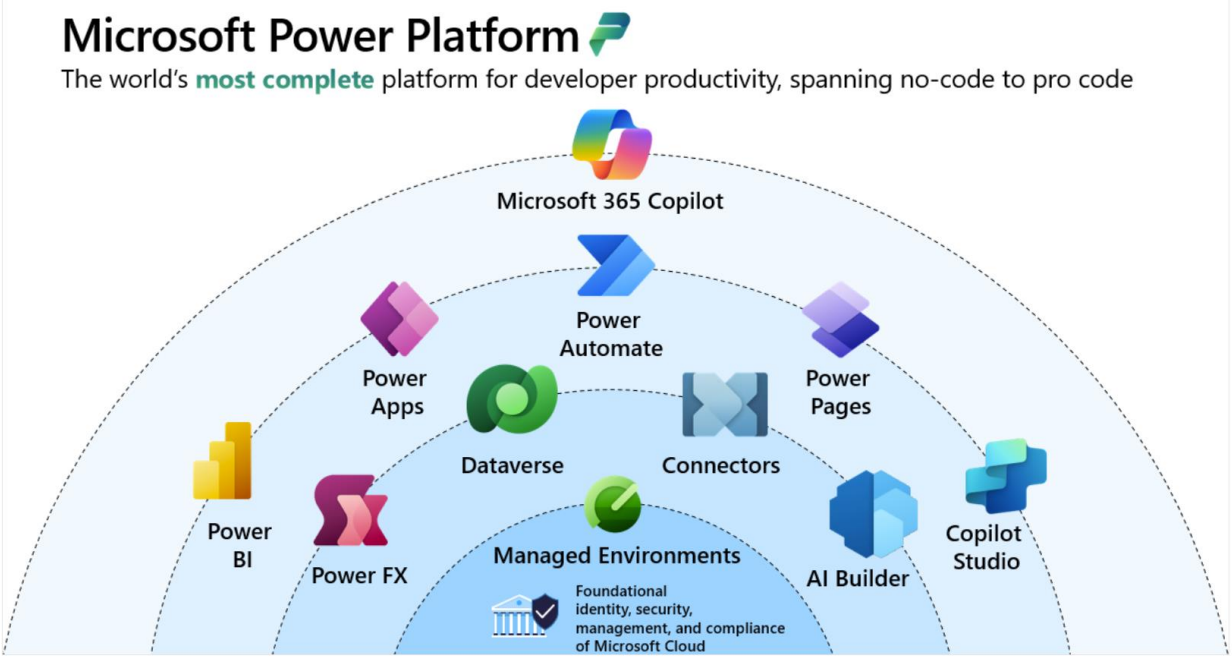
Power Platform & AI 2026

"Pure RPA" (Robotic Process Automation) refers to traditional, rule-based software robots that execute routine, repetitive digital tasks strictly using structured data and explicit logic, without the aid of artificial intelligence, machine learning, or cognitive decision-making.

Next Generation

Build and scale intelligent apps, agents, and automation with an AI platform that maintains enterprise-grade control, security, and trust.



Dataverse sits at the heart of this architecture. It's the platform's unified data layer: a governed, enterprise-grade foundation that AI capabilities are grounded in.

When an agent needs to look up a customer record, a prompt needs business context, or a model-driven app needs to surface smart suggestions, it reaches into Dataverse.

Contents

Course/Learn Path/ Modules

Robotic-process-automation-online-workshop

https://learn.microsoft.com/en-us/training/paths/robotic-process-automation-online-workshop/

Modules:

Install required software for Power Automate: Automation

https://learn.microsoft.com/en-us/training/modules/install-required-software-online-workshop/

- Get Power Apps Developer Plan
- Install the Power Automate for desktop application
- Register your machine
- Install the Coffee Shop Contoso invoice app (for lab scenario use)

Create your first desktop flow

https://learn.microsoft.com/en-us/training/modules/create-first-desktop-flow-online-workshop/

- Exercise- Get familiar with Power Automate for desktop
- Exercise - Create your first Power Automate desktop flow

Use input and output variables

https://learn.microsoft.com/en-us/training/modules/use-input-output-online-workshop/

- Exercise - Import the solution
- Exercise - Use input and output parameters

Integrate with cloud flows

https://learn.microsoft.com/en-us/training/modules/integrate-cloud-flows-online-workshop/

- Exercise - Create a cloud flow and a connection7 min
- Exercise - Monitor the list of machines5 min
- Exercise - Create a cloud flow with Describe it to design it20 min
- Exercise - Create a work queue

Use Outlook email to trigger desktop flows and pass inputs

https://learn.microsoft.com/en-us/training/modules/outlook-flows-online-workshop/

- Import the solution
- Exercise - Use an Outlook email to trigger a desktop flow and pass input

Add an AI model to process invoice forms

https://learn.microsoft.com/en-us/training/modules/add-ai-model-process-invoice-online-workshop/

- Exercise - Build and use AI models to enhance user experience in workflows

Integrate with Microsoft Teams to get approvals

- Exercise - Use Microsoft Teams to get approvals

Create subflows and web automation with Power Automate for desktop

https://learn.microsoft.com/en-us/training/modules/create-subflows-web-automation-online-workshop/

- Exercise - Build a Power Automate for desktop subflow to write notes into Excel15 min
- Exercise - Advanced Power Automate Desktop features

Migrate to New AI Dev

Get started with AI-first solutions in Microsoft Power Platform

https://learn.microsoft.com/en-us/training/paths/design-model-solutions-power-platform/

Build your data model with Microsoft Dataverse

https://learn.microsoft.com/en-us/training/paths/build-data-model-microsoft-dataverse/

Build intelligent apps and portals with Microsoft Power Apps

https://learn.microsoft.com/en-us/training/paths/build-apps-portals-power-apps/

Automate and extend your solutions with AI in Microsoft Power Automate

https://learn.microsoft.com/en-us/training/paths/automate-business-processes-power-automate/

Vocabulary Key Terms

1. AI Builder

Low-code AI capabilities in Power Platform used to process documents, predictions, text, images, and business data.

2. Approvals

A Power Automate feature that manages requests for people to approve or reject actions.

3. Outlook

Microsoft's email and calendar service often used as a trigger or action in flows.

4. Teams

Microsoft collaboration platform used for chat, meetings, approvals, notifications, and agent interactions.

5. Subflows

Reusable pieces of automation that can be called from other flows, reducing duplication.

6. Prompts

Instructions given to an AI model that define what response or action should be generated.

7. Plans

AI-powered planning capabilities that organize goals, tasks, and steps needed to complete work.

8. Dataverse

Microsoft's secure business data platform used to store and manage structured application data.

9. Decision Framework

A structured process used to determine the best automation, workflow, or AI solution for a scenario.

10. Flow vs Agent

A Flow executes predefined automation; an Agent interacts, reasons, and responds dynamically.

11. Desktop Flow vs Agent Action

A Desktop Flow automates UI tasks on applications; an Agent Action is a capability the agent invokes to perform work.

12. Dataverse

Centralized, secure data storage with tables, relationships, business rules, and security controls.

13. Model-Driven App Lab

A hands-on environment for building applications directly from Dataverse data and business processes.

14. Conversation Design

The practice of planning how users and AI interact through questions, responses, and actions.

15. Topics

Specific conversation areas that help an agent understand and respond to user requests.

16. Knowledge Sources

Documents, websites, SharePoint files, Dataverse tables, and other sources used to ground agent responses.

17. Actions

Tasks an agent can perform, such as calling flows, connectors, APIs, or business processes.

18. Governance

Policies, standards, and controls that manage how Power Platform solutions are built and used.

19. Security

Controls that protect data, users, applications, permissions, and access.

20. Input and Output Variables

Parameters passed into a flow or action (input) and values returned after execution (output).

21. Cloud Flows

Power Automate workflows that run in Microsoft's cloud using triggers and actions.

22. Desktop Flows

RPA automations that run on a desktop to interact with legacy applications and websites.

23. Approval Flow

A workflow that requests, tracks, and records approval decisions.

24. Grounded Prompts

AI prompts enhanced with trusted business data and knowledge sources to improve accuracy.

25. Copilot Studio Agent

An AI-powered assistant built in Microsoft Copilot Studio that can converse, answer questions, and perform actions.

Short Answers

1. When should you use a Flow?

Use a Flow when the process is predictable, rule-based, and requires automation without user conversation.

Example: Email notifications, approvals, file processing, data synchronization.

---

2. When should you use an Agent?

Use an Agent when users need conversational interaction, guidance, decision support, or AI-powered assistance.

Example: IT help desk, HR assistant, employee self-service bot.

---

3. When should an Agent call a Flow?

When a conversation needs to trigger business automation or perform work in other systems.

Example: Agent creates a ticket, submits a request, updates a record, or starts an approval.

---

4. When should Desktop Automation still exist?

When work must interact with legacy systems, desktop applications, or systems without APIs/connectors.

Example: Mainframe applications or old Windows software.

---

5. What is Manual Configuration vs Agent Builds?

Manual Configuration

- Human designs flows, topics, prompts, and actions.
- Greater control and customization.
- Best for complex business requirements.

Agent Builds

- AI helps create the agent automatically from natural language instructions.
- Faster development.
- Best for rapid prototyping and simple use cases.

Simple Interview Answer:

Manual Configuration is manually designing the solution step-by-step, while Agent Builds use AI to generate much of the agent automatically from requirements.

Compare & What is Changing (Retired PL-200/500)

From a **curriculum relevance** perspective, the issue isn't that **PL-200 content is bad**. It's that the *workflow for building solutions* has fundamentally changed.

The traditional PL-200 mindset was:

1. Gather requirements
2. Design Dataverse tables
3. Build app
4. Create flows manually
5. Test and deploy

The emerging Microsoft approach is:

1. Gather requirements
2. Use Copilot/Plan Designer to scaffold solution
3. Refine generated apps, agents, automations, and data model
4. Add AI orchestration and agent behaviors
5. Govern and operationalize

That shift is reflected much more strongly in **AB-410** than in PL-200. AB-410 explicitly focuses on building intelligent applications using AI, copilots, agents, Power Apps, Power Automate, Power Pages, and Copilot Studio together.

What I see happening from an instructional standpoint

The retired PL-200 exam measured competency in:

- Dataverse
- Model-driven apps
- Canvas apps
- Power Automate
- Security
- Integration

Those skills are still essential.

What's changing is **how students create those assets**.

Today, if you're teaching Power Automate and never show:

- Describe It to Design It

- Copilot flow creation
- Natural language flow generation
- Agent-triggered workflows
- Copilot Studio orchestration
- AI Builder integration

then students are learning a platform that no longer matches what they'll see in the product UI.

If I were redesigning a PL-200 era course today

I'd probably split it:

Foundation Layer (30%)

Still teach:

- Dataverse architecture
- Security roles
- Business process design
- Connectors
- Solution lifecycle

These concepts survive every UI change.

Modern Builder Layer (40%)

Focus on:

- Power Apps Copilot
- Flow generation from prompts
- Data model generation
- Plan Designer
- Agent-first design

This is where the platform is moving. The learning path you linked actually references Plan Designer and Copilot capabilities as part of the Power Platform story.

Intelligent Automation Layer (30%)

- Copilot Studio
- Agent actions
- Autonomous workflows

- Human-in-the-loop approvals
- Generative AI patterns
- Responsible AI governance

This is the gap most legacy PL-200 material has.

The closest current "successor" from a teaching perspective

If your goal is:

"What curriculum best matches how Microsoft expects solutions to be built in 2026?"

I'd argue:

AB-410 + Copilot Studio learning paths is closer than PL-200.

Not because it replaces PL-200's functional consultant knowledge, but because it reflects the current development experience.

My instructor recommendation

Keep the **PL-200 architecture topics**:

- Dataverse
- Security
- Solution management
- Connectors
- ALM

But replace much of the traditional "click-by-click flow building" labs with:

- Build a flow from a prompt
- Refine AI-generated expressions
- Create an agent that invokes a flow
- Use Copilot to generate tables and apps
- Add approval loops and governance

In other words, teach **how to supervise and improve AI-generated solutions**, not just how to manually build them from scratch.

That's where I see the industry demand moving, and AB-410 appears to be Microsoft's first formal course that acknowledges that reality rather than treating AI as an add-on

Nothing replaced the PL-500 labs one-for-one.

Microsoft retired PL-500 with no direct successor certification, and the training emphasis has shifted away from pure RPA toward AI agents, Copilot Studio, and intelligent application design. [d365training.com], [whizlabs.com]

If you look at the old PL-500 lab categories:

PL-500 Lab Topic	Modern Equivalent
Desktop Flows	Still Desktop Flows, but less emphasized
UI Automation	Power Automate + Agent actions
Browser Automation	APIs, connectors, agents
OCR Processing	AI Builder document processing
Invoice Automation	AI Builder + Power Automate + Agents
Human Approval Workflows	Agent + Flow + Approval patterns
Legacy System Automation	Hybrid of RPA and Copilot-generated automation
Bot Orchestration	Agent orchestration

The biggest change is that Microsoft is encouraging builders to ask:

"Can I use an API, connector, AI, or agent first?"

before asking:

"Can I automate the screen?"

What I'd use instead of PL-500 labs today

For instructor-led delivery, I'd build labs around:

1. Copilot-Assisted Flow Creation

Students:

- Describe a process in natural language
- Generate a cloud flow
- Refine actions
- Add error handling

This teaches the same automation thinking as PL-500 but using modern tooling.

2. AI Builder Document Processing

Classic PL-500:

- OCR invoice
- Extract fields
- Route approval

Modern version:

- AI Builder document model
- Extract structured data
- Agent summarizes exceptions
- Power Automate routes work

3. Agent Calls Flow

New pattern:

User

↓

Copilot Studio Agent

↓

Power Automate Flow

↓

SAP / SharePoint / ServiceNow

↓

Response

This is arguably the spiritual successor to many PL-500 orchestration labs.

4. Human-in-the-Loop Automation

Students build:

- Agent identifies issue
- Requests approval
- Executes flow
- Reports outcome

This preserves the strongest PL-500 concepts around exception handling.

The hidden gem from PL-500

Honestly, the most valuable thing wasn't Desktop Flows.

It was teaching to think in terms of:

1. Trigger
2. Decision
3. Action
4. Exception
5. Recovery
6. Monitoring

That mental model still applies perfectly to:

- Power Automate
- Copilot Studio
- AI Agents
- Azure AI workflows

The tool changed.

The process engineering skills did not.

If I were updating a PL-500 class in 2026

I'd keep about:

- 20% Desktop Flow labs
- 30% Cloud Flow labs
- 25% AI Builder labs
- 25% Copilot Studio / Agent labs

The closest thing to a "replacement lab set" is actually a blend of:

- **AB-410 Intelligent Applications**
- **Copilot Studio training**
- **AI Builder training**
- **Advanced Power Automate automation scenarios**

rather than a single course. Microsoft appears to have decomposed the old "RPA Developer" role into broader AI-enabled builder roles instead of maintaining a dedicated automation-developer track. , [whizlabs.com], [d365training.com]

As an instructor, I'd say **AB-410 replaces maybe 40-50% of PL-500's intent**, but the rest now lives in Copilot Studio and AI Builder content that wasn't part of the original PL-500 story.

Trial Accounts

The Microsoft Power Automate per-user with attended Robotic Process Automation (RPA) trial (now commonly integrated under the **Power Automate Premium** tier) is a 90-day evaluation (standard general trials are often 30 days, but self-assisted RPA trials run up to 90 days) that costs $0. It lets single users test desktop and cloud-based automated workflows, premium connectors, and user-assisted UI automation. [1, 2, 3, 4, 5, 6]

How to Start the Trial

- **From the Portal:** Go to the Power Automate Portal, navigate to **My flows** or **Desktop flows**, and click **Start free trial**.
- **From the App:** Open **Power Automate for desktop**, and click **Start trial** when prompted by premium feature notifications.
- **From Pricing:** Visit the official Power Automate Pricing page and select the trial option under the premium plans. [1, 2, 3, 4]

Key Trial Features

- **Attended RPA:** Run desktop flows on your machine while you watch or assist.
- **Cloud Flows & Connectors:** Build automated triggers and connect to paid enterprise tools.
- **Process Mining:** Test preview capabilities to analyze workflow bottlenecks. [1, 2, 3, 4, 5]

No, you cannot activate a Power Automate Premium trial using a standard personal @outlook.com email account, but **no credit card is required** to get the exact premium environment you need for that workshop. Microsoft requires a **work or school email address** to register for Power Automate premium trials. [1, 2, 3, 4]

As a student, you can easily bypass this limitation for free by using Microsoft's official developer backdoor to create a trial organization. [1]

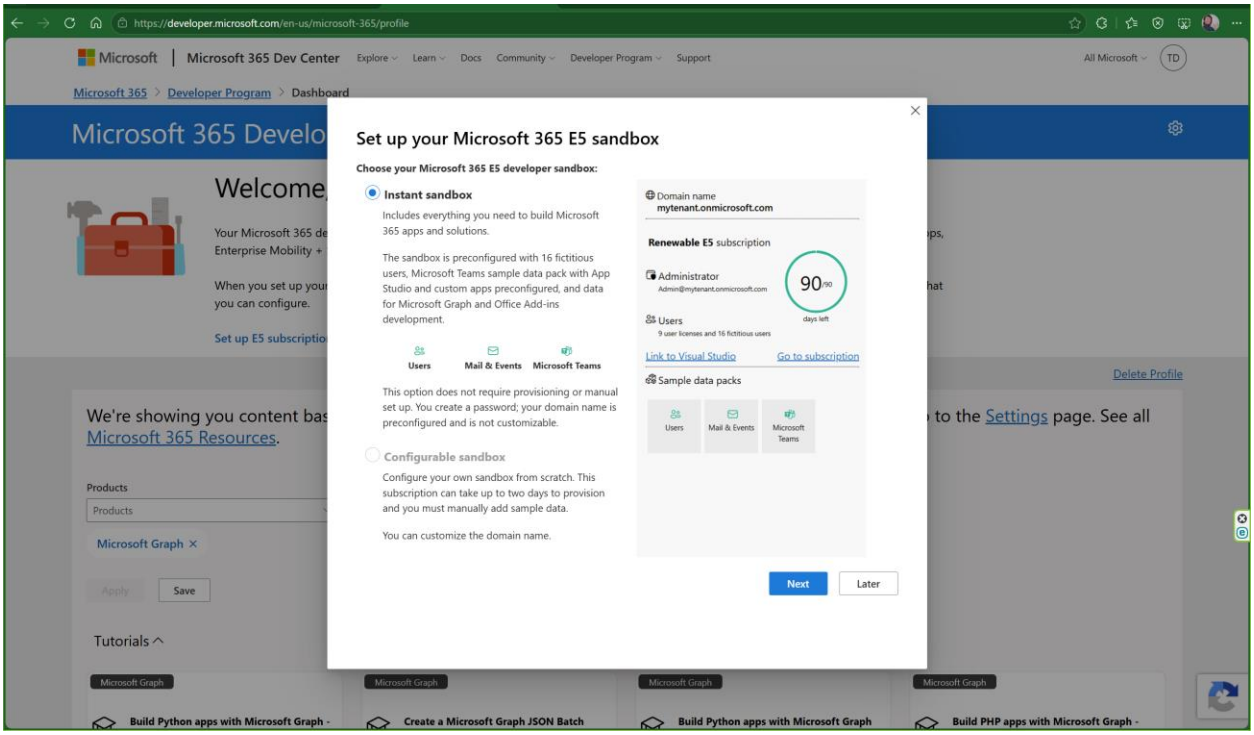
The Correct Workaround for Students

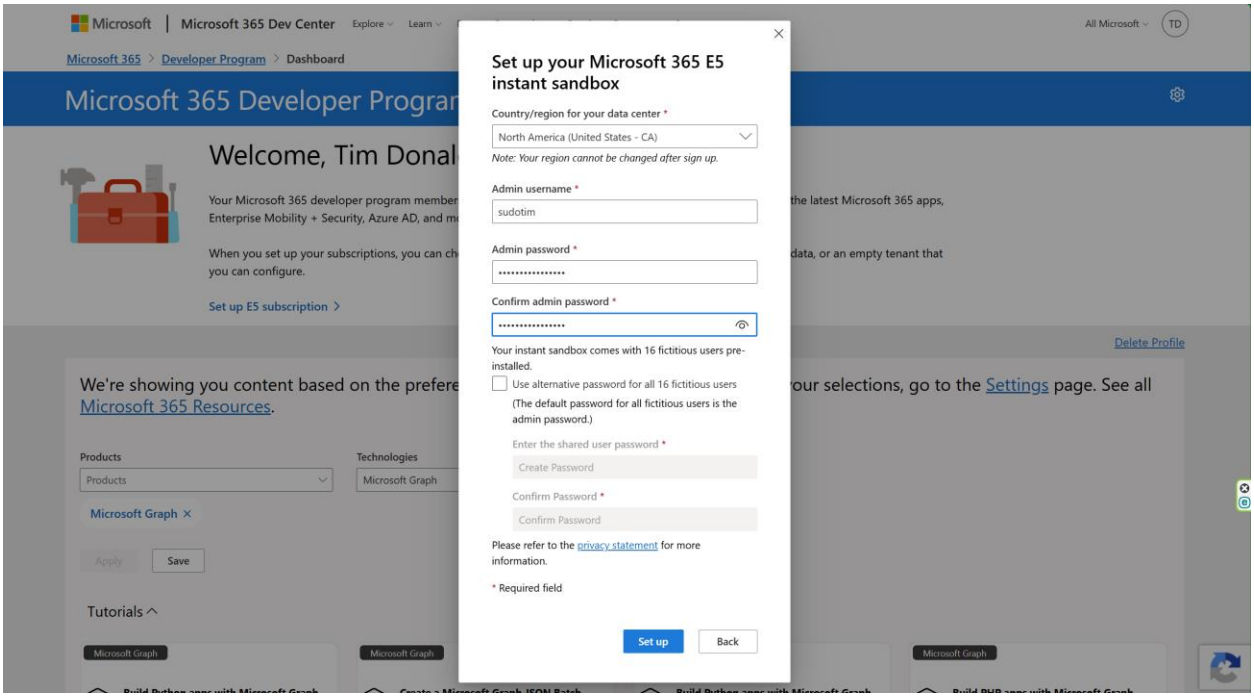
Install required software for Power Automate: Automation - Online workshop - Training | Microsoft Learn

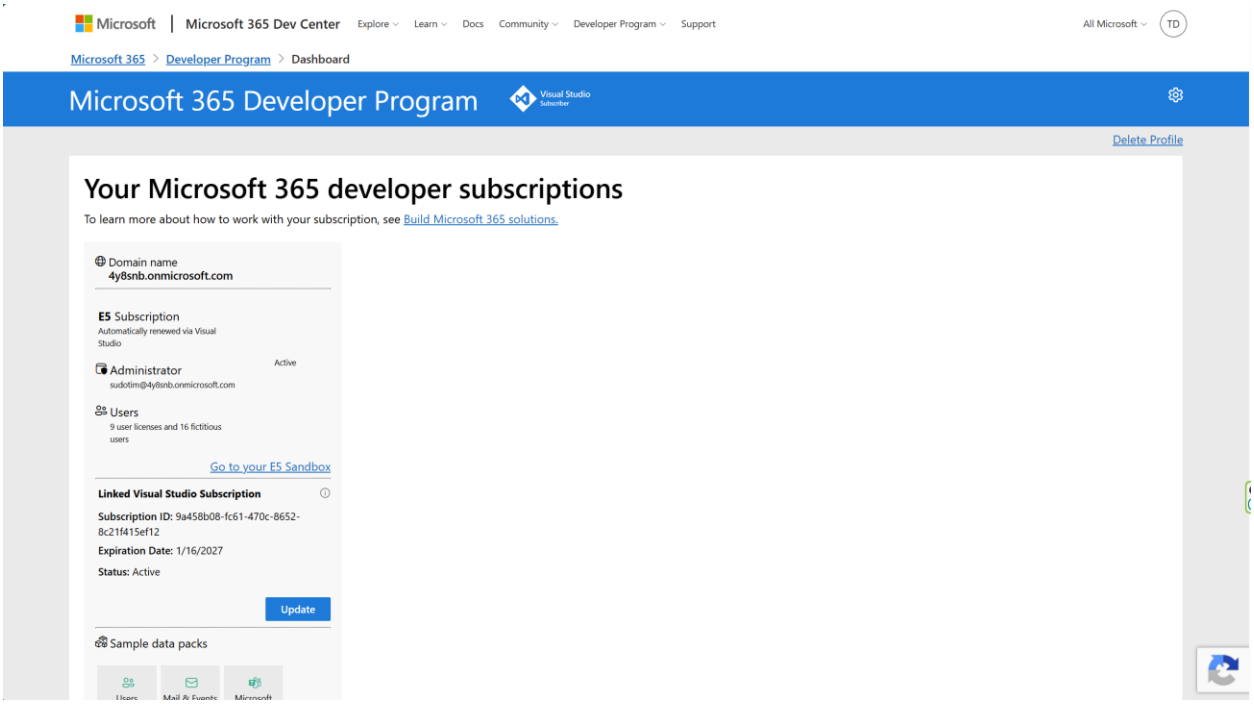
Instead of a personal Outlook account, you must create a free **Microsoft 365 Developer Account**. This gives you a free, fully functional enterprise "tenant" domain (e.g., yourname@yourdomain.onmicrosoft.com). [1, 2]

Downloading the Verification app for the login process is also required.

1. Go to the Microsoft 365 Developer Program and click **Join Now**.
2. Sign in with any free personal Microsoft/Outlook account just to register.
3. Follow the steps to set up a **Sandbox**. This will generate a brand new administrator email ending in .onmicrosoft.com.
4. No credit card is asked for during this process. [1, 2, 3, 4]







Activating the Power Automate Premium Trial

Once you have your new .onmicrosoft.com developer email address, you can officially satisfy the Install Required Software workshop prerequisites: [1]

- **Log In:** Go to the Power Automate Portal and sign in using your new developer .onmicrosoft.com credentials. [1, 2]
- **Start Trial:** Click on **Go Premium** or start a 90-day free trial from the settings gear icon inside the dashboard. Because it is a self-assisted developer trial, **no credit card is required**. [1, 2, 3, 4, 5]
- **Workshop Ready:** This environment automatically includes the required **Exchange Mailbox**, **Power Apps trial**, and **Dataverse Database** needed to finish your automation exercises. [1, 2]

Verify Premium

To verify that your **Power Automate Premium trial is successfully active** and ready for the workshop, you can check your status directly inside the web portal, the desktop application, or your developer admin center.

Method 1: Check the Web Portal (Fastest)

1. Navigate to the Power Automate Portal and log in with your developer email address.
2. Click the **Settings (gear icon)** in the top-right corner of the page.
3. Select **View my licenses** from the dropdown menu.
4. Under the **My licenses** list, you should explicitly see **Power Automate Trial** or **Power Automate Premium** listed with an active status. [1, 2, 3, 4]

Method 2: Check the Desktop App

1. Launch **Power Automate for desktop** on your computer.
2. Look at the top-right corner next to your profile icon and name.
3. If the trial is fully active, you will no longer see a **"Go Premium"** button or premium diamond icons locking down advanced steps (like cloud orchestration or work queues). [1, 2]

Method 3: Check your M365 Admin Hub

Since you created a developer sandbox organization, you can view the exact backend licenses assigned to you:

1. Go to the Microsoft 365 Admin Center.
2. On the left navigation pane, expand **Billing** and select **Licenses**.
3. Look for **Power Automate Premium Trial** or **Power Automate trial** under your active subscription items. [1, 2, 3]

Note on Trial Accounts & Admin Area

Q - Power apps license area shows MS 365 e5 Developer but Power Automate license area is blank. Are they different?

Yes, they are completely different.

The **Microsoft 365 E5 Developer** license covers standard productivity tools (like Outlook, Teams, and Excel) and includes a *limited* seed version of Power Apps and Power Automate. However, it **does not include Power Automate Premium** or Robotic Process Automation (RPA) rights. [1, 2, 3, 4, 5]

This explains why your Power Apps area shows the E5 plan, but your Power Automate license area is blank. You have the basic Office automation rights, but you have not yet activated the specific Premium standalone trial required for your workshop. [1, 2, 3, 4]

How to Fix It and Fill the Blank Area

To get your Power Automate Premium trial running on your new Developer account, follow these quick steps:

1. **Trigger the Trial:** Go to the Power Automate Portal and ensure you are logged into your new developer .onmicrosoft.com account.
2. **Start the Process:** Click on **My flows** in the left sidebar, then click **New flow** -> **Instant cloud flow** or **Desktop flow**.
3. **Accept the Prompt:** Try to add a **Premium action** or click the **"Go Premium" / "Start Free Trial"** prompt that pops up on your screen.
4. **Verify Again:** Once you accept that trial prompt, refresh your licensing area. You will see **Power Automate Premium Trial** populate that blank space. [1, 2]

Don't worry—activating this trial will remain completely free and won't require a credit card since you are working inside an isolated developer sandbox environment.

Steps to Create Premium Trial after creating Developer Environment

1. Click Start trial link
2. Create Dataverse database

Error

Q - does error - Attempted to get endpoint for CDS when there is no linked metadata for the environment - creating dataverse common

Yes, this specific error means that your Power Automate actions are trying to pull or push data to Dataverse (formerly called CDS - Common Data Service), but the Power Platform environment you are currently in **does not have a Dataverse database created or linked to it**. [1, 2]

Because you are using a brand-new Microsoft 365 Developer environment, it gives you an automated cloud platform, but by default, it leaves the database unprovisioned. When Power Automate tries to fetch data layout tables (metadata), it hits a brick wall and throws the endpoint failure. [1, 2]

You can fix this issue inside your developer sandbox without restarting your work.

Refresh page F5

How to Create the Database & Resolve the Error

1. **Open the Admin Hub:** Go to the Power Platform Admin Center and log in with your developer account.
2. **Select Your Environment:** Click on **Environments** in the left navigation sidebar. Click on your active workshop environment name (if you haven't created a custom one, select your **Developer / Personal** environment).
3. **Add Database:** Look at the top menu bar or the **Details** pane on the screen. Look for the **Dataverse / Database status**. Click the button that says **Add database** (or **Create Database**). [1]
4. **Configure Settings:**
    - Select your preferred **Currency** and **Language**.
    - Toggle **Deploy sample apps and data** to **Yes** (this is highly recommended for student workshops so you have fake data to run tests on).
5. **Save and Wait:** Click **Save**. The system will take 2 to 5 minutes to provision the database infrastructure in the background. [1]

What to Do After Provisioning

Once the database status switches to **Ready** in the admin center, return to your cloud flow in the Power Automate Portal. [1, 2]

Open your workshop flow, click **Edit**, and delete then re-add the specific Dataverse step that threw the error. This refreshes the metadata link, establishes the correct endpoint connection, and stops the error from recurring.

Next Step: Contoso Coffee Shop Files

About Contoso:

Map Business Processes to AI-First Solution Patterns - Training | Microsoft Learn

Map Business Processes

Map Business Processes to AI-First Solution Patterns - Training | Microsoft Learn

Install Files

Install the Coffee Shop Contoso invoice app (for lab scenario use) - Training | Microsoft Learn

- Download & Unzip in your own folder

The Modernization Journey:

Manual Process → Flow → RPA → AI Extraction → Human Approval → AI Agent.

What are the Automation in a Day files?

The Automation in a Day learning path includes downloadable exercise files and labs used throughout the workshop. They provide a complete sample business process that students automate end-to-end in Power Automate.

User Perspective

Think of it as:

- A fake company (Contoso)
- A fake invoice processing process
- Sample applications, invoices, and data
- A safe environment to learn automation without impacting real systems

The learner can focus on **how automation works** rather than worrying about business data or application setup.

What does the Contoso Invoicing Setup bring to the table?

The Contoso scenario simulates a common business problem:

"Invoices arrive by email and someone manually enters them into a business application."

Across the labs, students progressively automate that process by:

1. Creating a desktop flow

2. Using input/output variables
3. Connecting desktop and cloud flows
4. Triggering automation from Outlook
5. Extracting invoice data with AI Builder
6. Sending approvals through Teams
7. Recording results in Excel and web applications

User Perspective

A business user sees:

Before	**After**
Open email	Flow detects email
Read invoice	AI extracts data
Re-key information	Desktop flow enters data
Ask manager	Teams approval
Update spreadsheet	Automated update

The learner experiences a complete business process modernization journey.

Good catch. Looking through the instructor deck, there are actually a few clues.

What the deck explicitly tells us

The solution architecture consistently refers to the Contoso application as:

"Legacy invoice management software" / "Contoso Invoicing Application" that Power Automate for desktop opens and fills in. [Automation...uctor Deck | PDF]

The architecture diagram shows:

- Power Automate Desktop
- Contoso Invoicing Application
- Excel (for audit logging)
- Currency conversion website
- Outlook / Teams / AI Builder

But **no database component is shown behind the Contoso app**. [Automation...uctor Deck | PDF]

The lab instructions also describe the desktop flow as:

opening the legacy Win32 application and filling in invoice details. [Automation...uctor Deck | PDF]

What that strongly suggests

For training purposes Microsoft typically wanted:

- deterministic screens
- repeatable results
- no external dependencies
- easy installation

Because of that, the app is almost certainly one of:

1. **Local file storage** (XML/JSON/flat file)
2. **Embedded database** (SQLite, SQL CE, LocalDB)
3. **In-memory demo records**

I would place my bet on **SQLite or a local data file**, but that's an educated guess, not something stated in the materials.

The bigger clue

Notice what students are taught to automate:

- Read invoice
- Enter values
- Save invoice
- Capture output values
- Read records back
- Use Process Advisor to record user interactions with the application [Automation...uctor Deck | PDF]

At no point are students:

- querying a database
- modifying schemas
- connecting through an API
- integrating directly with the application's data layer

That's intentional.

The entire purpose is:

"Treat the application as a black box and automate it through the UI."

That's classic PL-500 RPA thinking. [Automation...uctor Deck | PDF]

If you're trying not to reinvent the wheel

For a workshop called:

Modernizing Power Automate: From RPA and Flows to AI Agents

I would not spend any time reproducing the storage mechanism.

Instead I'd tell the audience:

Stage	System
Legacy	Contoso Win32 Invoice App
RPA	Desktop Flow enters data
Workflow	Cloud Flow orchestrates
AI	AI Builder extracts invoice details
Human Oversight	Teams Approval
Modernization	AI Agent coordinates process

The Contoso application becomes a **legacy system placeholder**. The actual persistence technology behind it is almost irrelevant to the learning objectives, which is why Microsoft never really talks about it in the deck. [Automation...uctor Deck | PDF]

If you have the student download package

A quick way to know for sure:

Look in the Contoso Invoicing installation folder for:

- *.db
- *.sqlite
- *.mdf
- *.sdf
- *.xml
- *.json

or run Process Monitor/File Monitor while saving a record.

That will reveal the storage mechanism in a minute or two. The instructor deck itself doesn't document it.

Honestly, for your modernization narrative, I'd treat the app as**"Legacy LOB App (intentionally opaque)"**, because that's exactly the mindset Microsoft was teaching in the Automation in a Day RPA workshops.

How this maps to the old PL-500 story

Then (PL-500 RPA Developer)

Focus:

- UI automation
- Desktop flows
- Attended RPA
- Unattended RPA
- Legacy applications
- Screen scraping
- Process automation

The message was:

"Automate what people do."

Now (Modern Power Automate + AI)

Focus:

- Cloud flows
- AI Builder
- Copilot
- Intelligent document processing
- AI Agents
- Human-in-the-loop approvals
- Business process orchestration

The message becomes:

"Automate decisions and outcomes, not just clicks."

The Contoso invoicing lab actually tells that evolution story very well because it starts with traditional RPA and gradually introduces AI and approvals.

Suggested Training Transition

Day 1-2

PL-500 / Power Automate RPA Foundations

Use the Automation in a Day labs:

- Desktop flows
- Variables
- Cloud-to-desktop integration
- Outlook triggers
- Teams approvals

These provide the "how automation works" foundation.

Hand Off

Move to:

AB-410T00-A: Build Intelligent Applications

Position it as:

Traditional Automation	**Intelligent Automation**
Rules	AI reasoning
Desktop flows	AI Agents
Data entry	Information extraction
If/Then logic	Natural language understanding
Process execution	Decision support

Summarize it as:

Automation in a Day teaches how to automate work.

Contoso Invoicing provides the realistic business scenario.

PL-500 taught RPA and workflow automation.

AB-410 extends that foundation into AI-powered applications and agents.

The modernization journey is: Manual Process → Flow → RPA → AI Extraction → Human Approval → AI Agent.

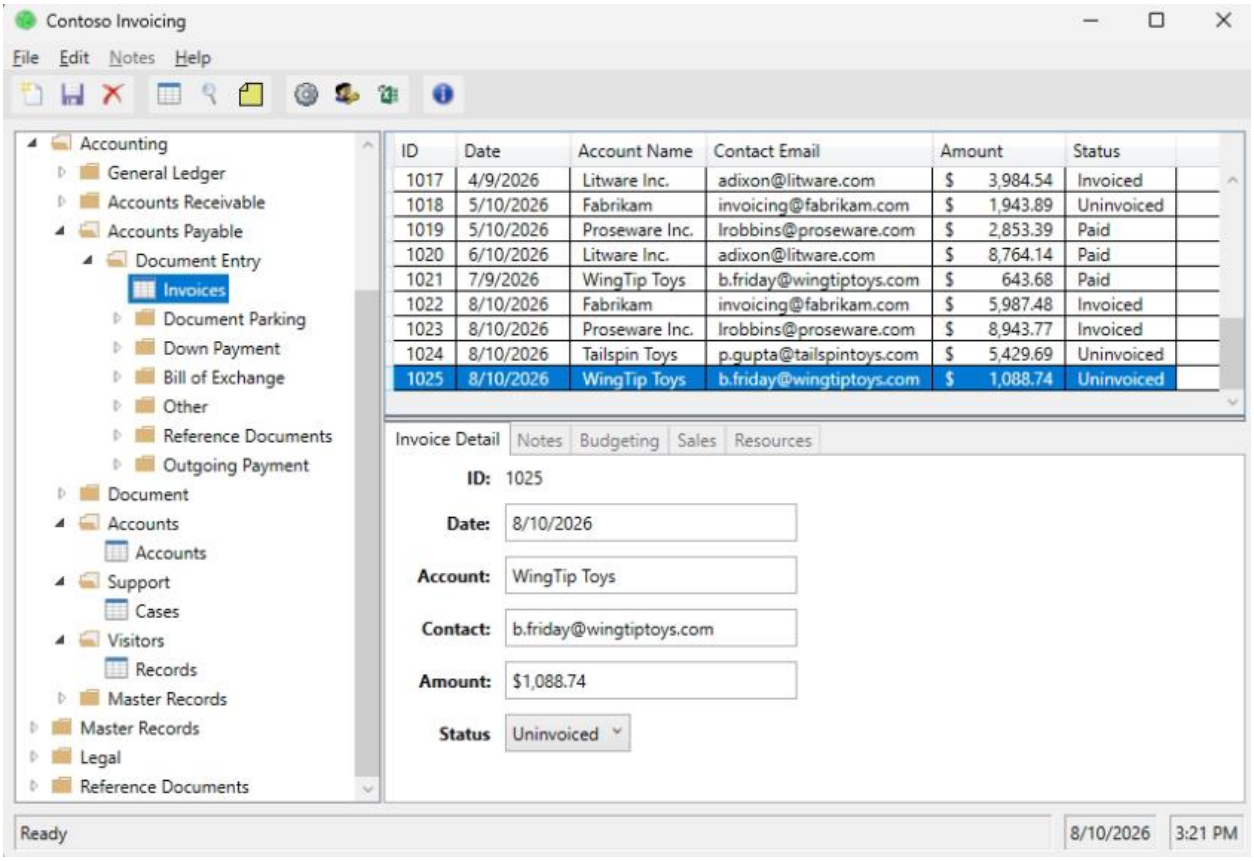
That story aligns extremely well with the current "Modernizing Power Automate: From RPA and Flows to AI Agents" narrative because the Contoso invoicing scenario already contains most of the stepping stones needed to explain the evolution.

Contoso Invoicing app functionality

Does the Contoso Invoicing app have any function other than display a set of records that is exercised by the user? looking at the app only not the flow instructions just to fine point what to expect from it.

Short answer: **No, the Contoso Invoicing application is not intended to be a fully functional business application.** Its primary purpose is to provide a predictable desktop UI that Power Automate for desktop can interact with and automate.

"Legacy LOB App (intentionally opaque)"



From the labs, the app is used to:

- Display invoice records
- Open/create invoice entries
- Accept invoice field inputs (vendor, amount, invoice number, dates, etc.)

- Save the entered data
- Return values that can be captured as outputs by a desktop flow

What a learner actually experiences

When you launch the app manually, it generally feels like a small Windows line-of-business application:

- List of invoices/records
- Forms with text boxes and buttons
- Ability to create or enter invoice information
- Ability to save/update a record

These actions exist mainly so the desktop flow recorder has something realistic to click, type into, read from, and validate. ,

What it is *not*

The app is not intended to demonstrate:

- ERP functionality
- Accounting processes
- Invoice matching
- Payment processing
- Vendor management
- Business rules engines
- Dataverse integration

Those capabilities are modeled by the **Power Automate solution around the app**, not by the app itself.

For your modernization narrative

I would characterize the Contoso Invoicing app as:

"A stand-in for a legacy desktop application."

Its value is that it gives students something that looks and behaves like the kind of legacy app organizations still have today.

So the story becomes:

1. Human enters invoice into Contoso app.
2. Desktop Flow automates the same entry.
3. Cloud Flow orchestrates the process.

4. AI Builder extracts invoice data.
5. Teams approvals introduce human oversight.
6. AI Agents become the next evolution.

In that story, **the Contoso app itself never becomes the star of the show**. It's simply the legacy system that the automation, AI, and agent technologies are built around.

Hosted Machine (optional)

For the **Automation in a Day / Online Workshop labs**, skipping the **Hosted Machine (optional)** step typically has **no impact on the lab outcomes** as long as you have:

- Power Automate for desktop installed locally
- Your machine registered
- A desktop flow connection configured
- The Contoso Invoicing app installed locally

In that setup, all desktop automation runs on **your own workstation** rather than a Microsoft-hosted machine.

What changes?

Option	Where Desktop Flow Runs
Local machine	Your PC
Hosted machine (optional)	Microsoft-managed cloud PC

What doesn't change?

The following lab activities work the same way:

- Recording desktop flows
- Automating the Contoso Invoicing application
- Passing inputs and outputs
- Calling desktop flows from cloud flows
- Outlook-triggered automation
- AI Builder invoice extraction
- Teams approvals
- Excel/web automation exercises

The business scenario and flow design remain unchanged.

Why Microsoft includes it as optional

The hosted machine demonstrates a more enterprise-oriented deployment model:

- No dependency on a learner's workstation being powered on
- Better alignment with unattended automation scenarios
- Easier to discuss scaling and central management
- Supports the "digital worker" concept

For an Automation in a Day attendee, the learning goal is usually:

"Can I build and run a desktop flow?"

not

"Can I operate a cloud-hosted RPA infrastructure?"

For your modernization storyline

I'd actually skip it unless you specifically want to discuss:

RPA Evolution

1. User desktop (attended)
2. Dedicated VM (unattended)
3. Hosted machine
4. Cloud orchestration
5. AI-enhanced automation
6. AI Agents

For your **"From RPA and Flows to AI Agents"** narrative, the hosted machine is mostly an infrastructure detail. The core learning journey around Contoso Invoicing, cloud flows, AI Builder, and approvals remains intact without it. The main difference is simply **whose computer is clicking the buttons**.

Attended vs Unattended

The practical line in the sand is actually much simpler than the licensing diagrams make it appear:

Attended RPA

A human is involved in starting or supervising the automation.

Characteristics:

- User is signed into Windows.
- User session is active.
- Flow can interact with a user's desktop.

- User may click a button, start a cloud flow, or otherwise initiate the process.
- The robot and human effectively share the same workstation. [Automation...uctor Deck | PDF]

In the Automation in a Day labs, most of what you're doing is essentially attended RPA:

- Open Contoso Invoicing
- Record actions
- Test playback
- Run from your machine
- Watch the automation execute [Automation...uctor Deck | PDF]

Unattended RPA

The automation runs without a human actively using the machine.

Characteristics:

- User does **not** need to be logged in.
- Cloud flow sends work to a machine.
- Machine automatically signs in using stored credentials.
- Desktop flow runs independently.
- Results are returned to Power Automate. [Automation...uctor Deck | PDF]

The instructor deck explicitly describes unattended as:

"Accelerate the automation of high-volume and tedious tasks without lifting a finger" and jobs execute on dedicated machines rather than requiring a signed-in user. [Automation...uctor Deck | PDF]

Where people get confused

A cloud flow triggering a desktop flow **does not automatically make it unattended**.

For example:

Email arrives

↓

Cloud Flow starts

↓

Desktop Flow runs on Tim's laptop

↓

Tim is logged in

That's still fundamentally attended.

Now compare:

Email arrives

↓

Cloud Flow starts

↓

Dedicated VM wakes up

↓

Signs in automatically

↓

Desktop Flow runs

That's unattended.

The trigger isn't the deciding factor.

The deciding factor is:

Does a user session need to be present and available?

Why Microsoft split the licensing

Historically Microsoft viewed these as different business scenarios:

Attended

Personal productivity

Examples:

- Finance analyst
- HR coordinator
- Operations specialist

Automation assists a worker.

Unattended

Digital worker

Examples:

- Invoice processing queue
- Nightly ERP updates
- Order entry
- Claims processing

Automation replaces a human execution step. [Automation...uctor Deck | PDF]

That's why unattended capacity has traditionally commanded higher licensing costs.

For your modernization presentation

I'd update the old PL-500 language into something like:

Era	Primary Capability
Attended RPA	Human + Bot
Unattended RPA	Bot runs process
Cloud Flows	Process orchestration
AI Builder	Understand content
AI Agents	Make decisions and coordinate work

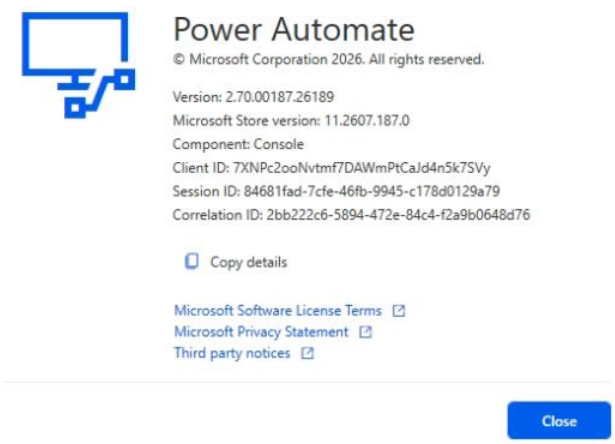
The key transition is:

Attended and unattended RPA automate tasks.

AI Agents automate outcomes.

That's probably the cleanest line to draw when handing students from the old PL-500 mindset into AB-410's intelligent applications and agent-centric architecture.

The First Flow



How the Entire Solution Works

Step 1

Email arrives.

```
Outlook Trigger
```

Step 2

Invoice PDF attached.

```
AI Builder
```

extracts:

```
Vendor
Amount
Invoice Number
Date
```

Step 3

Cloud Flow receives extracted values.

Variables:

```
Amount
Contact
Account
```

Step 4

Cloud Flow calls Desktop Flow.

Passes:

```
Amount
Contact
Account
```

Step 5

Desktop Flow performs UI automation.

```
Open Accounting System

Populate Fields

Save
```

Step 6

Application generates:

```
InvoiceID
```

Desktop Flow returns:

```
InvoiceID
```

as an Output Variable.

Step 7

Cloud Flow receives:

```
InvoiceID
```

Step 8

Approval

```
Teams
Approval Action
```

Human reviews invoice.

Step 9

If approved:

```
Subflow
```

updates Excel.

Excel Subflow Example

Inputs:

```
InvoiceID
Amount
ExchangeRate
```

Logic:

```
ConvertedAmount =
Amount * ExchangeRate
```

Outputs:

```
ConvertedAmount
```

Actions:

```
Write row to Excel

InvoiceID
Original Amount
Converted Amount
Approval Status
```

When designing variables ask:

```
1. Where does the value come from?
2. Who changes it?
3. How long do I need it?
4. Does it move between flows?
5. Does it change between environments?
```

- If it changes between Dev/Test/Prod → **Environment Variable**
- If it exists only during execution → **Flow/Desktop Variable**
- If it comes into a Desktop Flow → **Input Variable**
- If it leaves a Desktop Flow → **Output Variable**
- If one automation calls another → think **Parameter** and **Return Value**, just like a function in programming.

User Tasks & The Citizen Developer

Is the Capture UI, Image etc, basically doing basic windows legacy type user interactions, click buttons etc? Yes

- walk the UI prompts
- record steps automate create
- use variables
    - create templates or not
- Capture UI Elements
- Capture Images
- Debug
- Publish

Review, Edit, Customize w Copilot, AI Builder

AI Builder acts as a backend intelligence engine for processing data (like extracting text or invoices), whereas Copilot Studio builds front-end conversational chatbots and autonomous agents that can interact with users and execute business logic

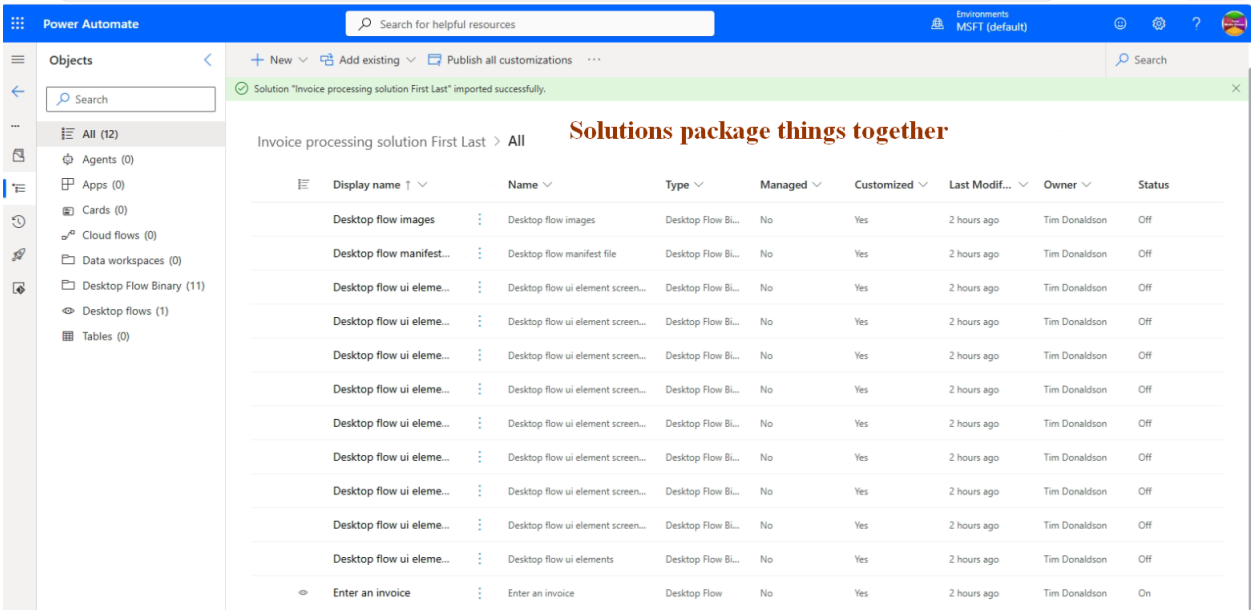
Identify which steps belong to which pattern and design the handoff points between them carefully.

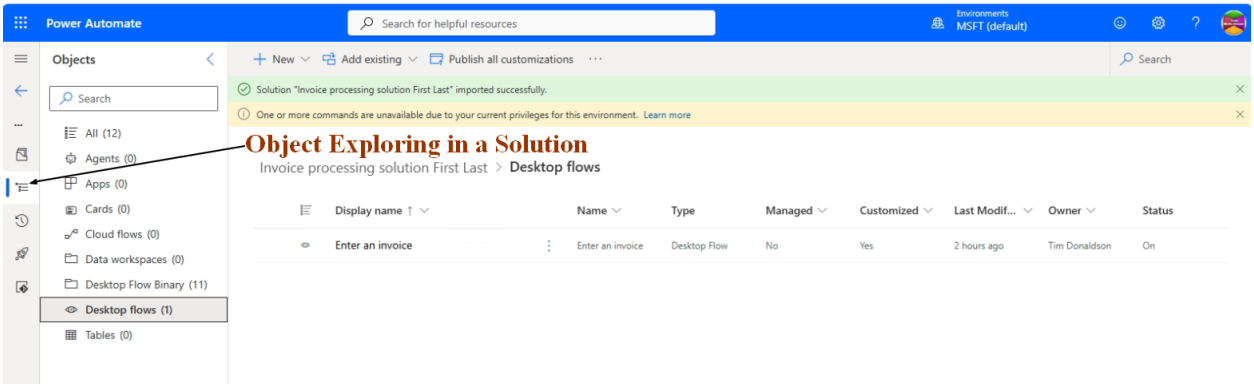
- Which steps require human effort?
- Repetitive, rule-based, and information-intensive (automatioin)

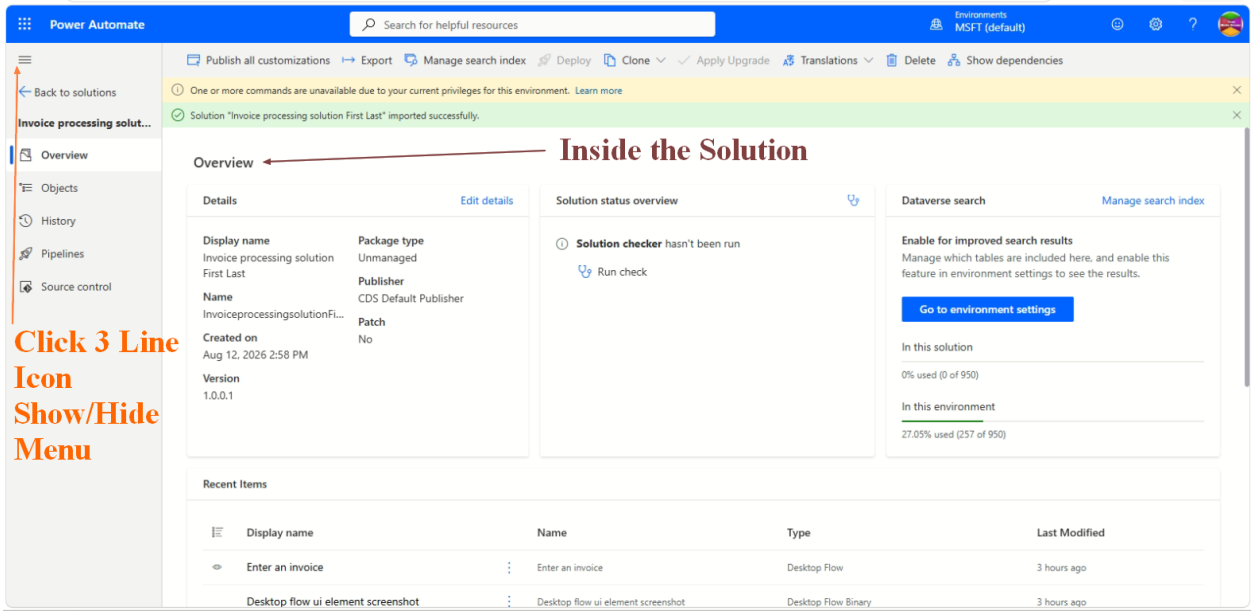
vs Judgment, Empathy, or Accountability - Human

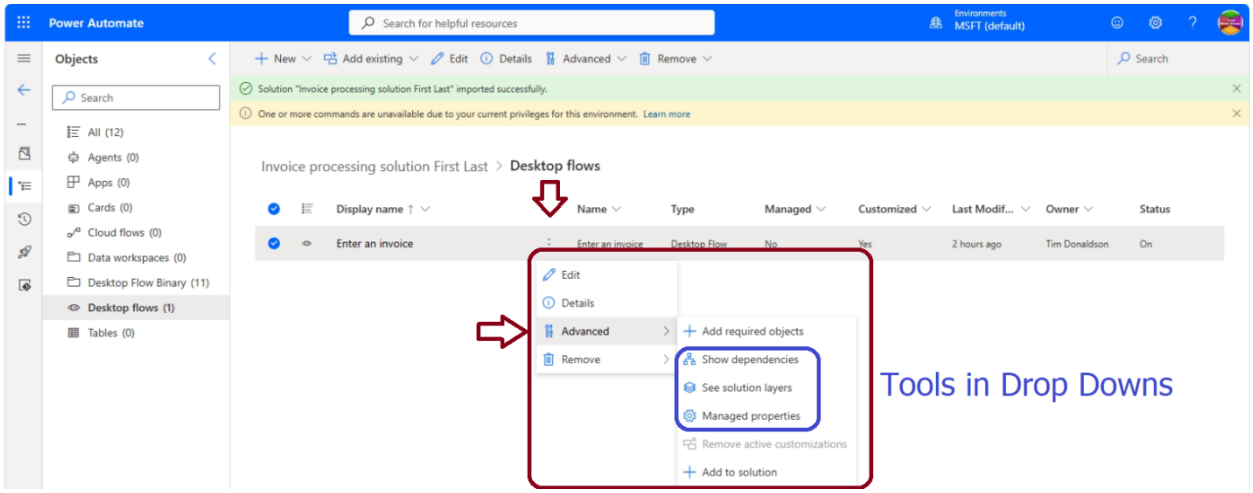
- Start with the agents Microsoft already provides

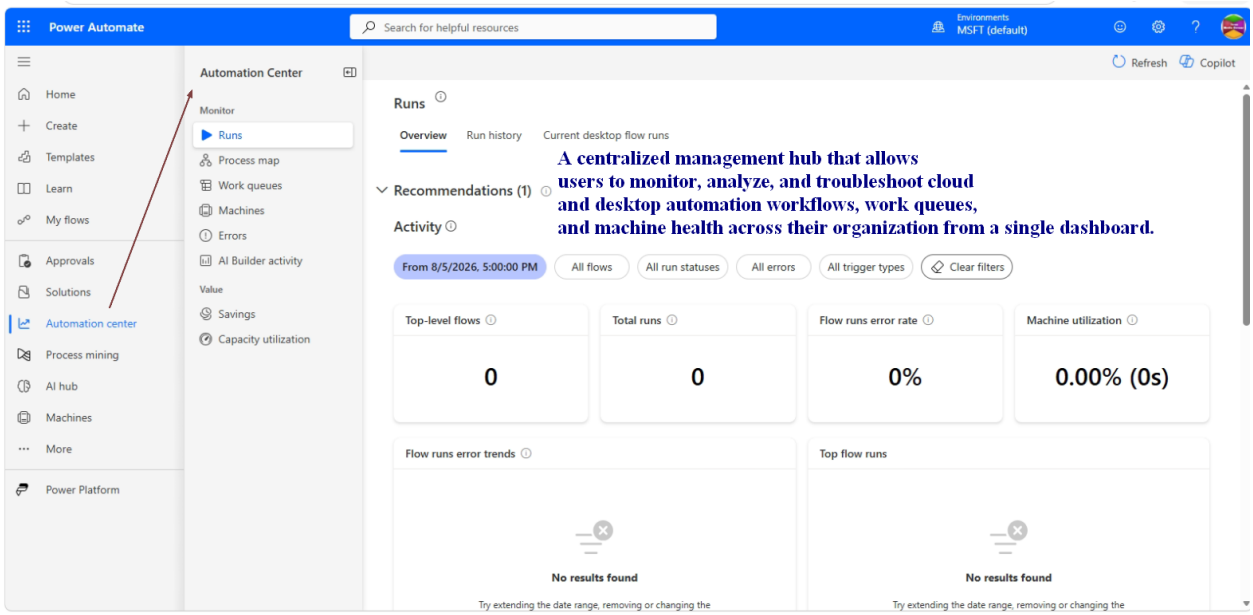
Solutions, Packaging, Screenshots



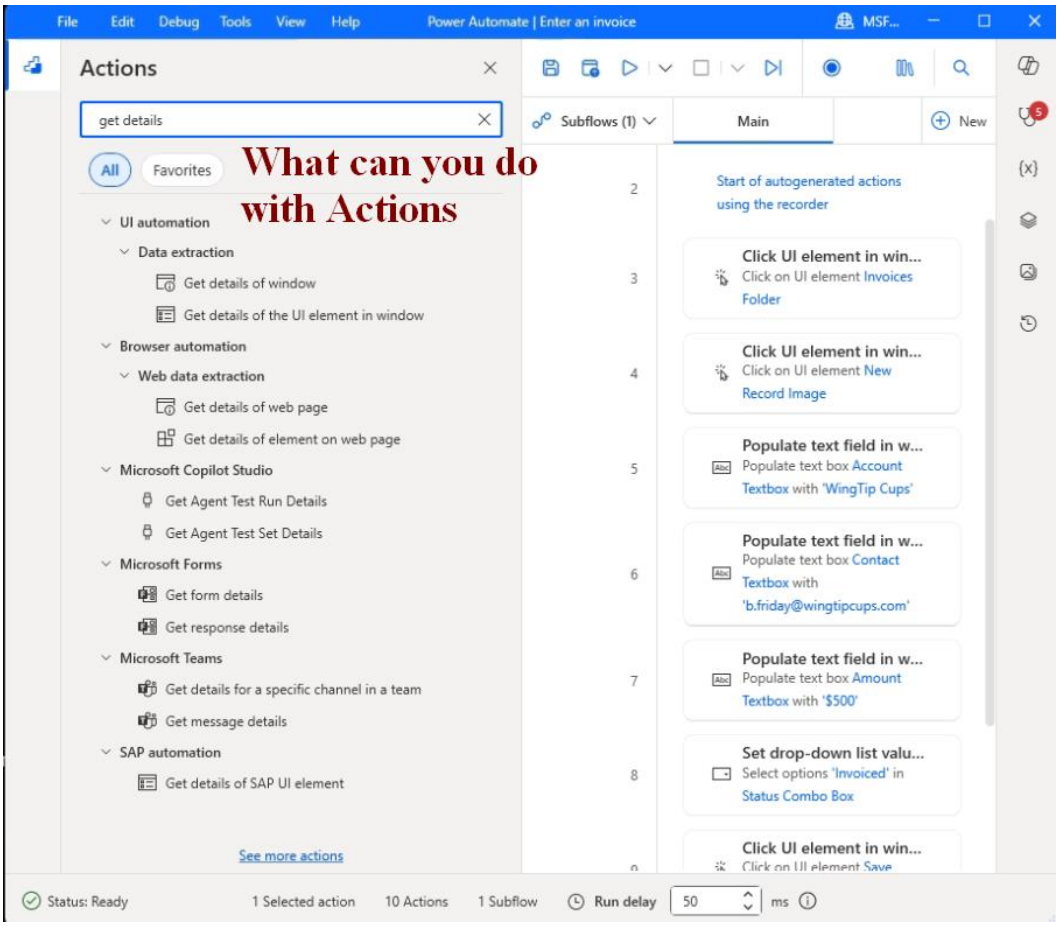

Power Automate
Search for helpful resources
Environments
MSFT (default)
Objects
Search
All (12)
Agents (0)
Apps (0)
Cards (0)
Cloud flows (0)
Data workspaces (0)
Desktop Flow Binary (11)
Desktop flows (1)
Tables (0)
New
Add existing
Publish all customizations
Solution "Invoice processing solution First Last" imported successfully.
One or more commands are unavailable due to your current privileges for this environment. Learn more
Object Exploring in a Solution
Invoice processing solution First Last > Desktop flows
Display name
Name
Type
Managed
Customized
Last Modif...
Owner
Status
Enter an invoice
Enter an invoice
Desktop Flow
No
Yes
2 hours ago
Tim Donaldson
On


Power Automate
Search for helpful resources
Environments
MSFT (default)
Publish all customizations
Export
Manage search index
Deploy
Clone
Apply Upgrade
Translations
Delete
Show dependencies
Back to solutions
Invoice processing solut...
Overview
Objects
History
Pipelines
Source control
One or more commands are unavailable due to your current privileges for this environment. Learn more
Solution "Invoice processing solution First Last" imported successfully.
Overview
Inside the Solution
Click 3 Line Icon Show/Hide Menu
Details
Edit details
Display name
Invoice processing solution First Last
Package type
Unmanaged
Name
InvoiceprocessingsolutionFi...
Publisher
CDS Default Publisher
Created on
Aug 12, 2026 2:58 PM
Patch
No
Version
1.0.0.1
Solution status overview
Solution checker hasn't been run
Run check
Dataverse search
Manage search index
Enable for improved search results
Manage which tables are included here, and enable this feature in environment settings to see the results.
Go to environment settings
In this solution
0% used (0 of 950)
In this environment
27.05% used (257 of 950)
Recent Items
Display name
Name
Type
Last Modified
Enter an invoice
Enter an invoice
Desktop Flow
3 hours ago
Desktop flow ui element screenshot
Desktop flow ui element screenshot
Desktop Flow Binary
3 hours ago

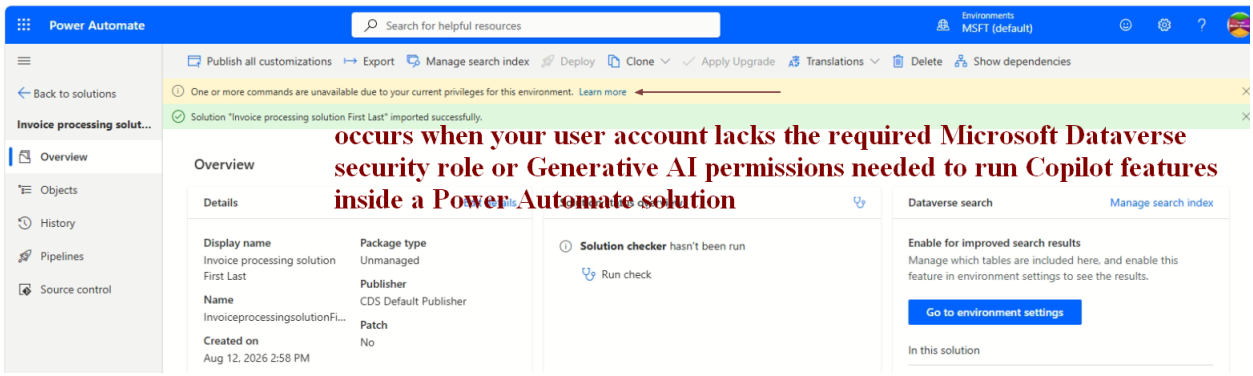

Power Automate
Search for helpful resources
Environments
MSFT (default)
Objects
Search
All (12)
Agents (0)
Apps (0)
Cards (0)
Cloud flows (0)
Data workspaces (0)
Desktop Flow Binary (11)
Desktop flows (1)
Tables (0)
New
Add existing
Edit
Details
Advanced
Remove
Solution "Invoice processing solution First Last" imported successfully.
One or more commands are unavailable due to your current privileges for this environment. Learn more
Invoice processing solution First Last > Desktop flows
Display name
Name
Type
Managed
Customized
Last Modif...
Owner
Status
Enter an invoice
Enter an invoice
Desktop Flow
No
Yes
2 hours ago
Tim Donaldson
On
Edit
Details
Advanced
Remove
Add required objects
Show dependencies
See solution layers
Managed properties
Remove active customizations
Add to solution
Tools in Drop Downs


Power Automate
Search for helpful resources
Environments
MSFT (default)
Refresh
Copilot
Home
Create
Templates
Learn
My flows
Approvals
Solutions
Automation center
Process mining
AI hub
Machines
More
Power Platform
Automation Center
Monitor
Runs
Process map
Work queues
Machines
Errors
AI Builder activity
Value
Savings
Capacity utilization
Runs
Overview
Run history
Current desktop flow runs
A centralized management hub that allows
users to monitor, analyze, and troubleshoot cloud
and desktop automation workflows, work queues,
and machine health across their organization from a single dashboard.
Recommendations (1)
Activity
From 8/5/2026, 5:00:00 PM
All flows
All run statuses
All errors
All trigger types
Clear filters
Top-level flows
0
Total runs
0
Flow runs error rate
0%
Machine utilization
0.00% (0s)
Flow runs error trends
No results found
Top flow runs
No results found

Actions


File
Edit
Debug
Tools
View
Help
Power Automate | Enter an invoice
Actions
get details
All
Favorites
What can you do
with Actions
UI automation
Data extraction
Get details of window
Get details of the UI element in window
Browser automation
Web data extraction
Get details of web page
Get details of element on web page
Microsoft Copilot Studio
Get Agent Test Run Details
Get Agent Test Set Details
Microsoft Forms
Get form details
Get response details
Microsoft Teams
Get details for a specific channel in a team
Get message details
SAP automation
Get details of SAP UI element
See more actions
Subflows (1)
Main
New
Start of autogenerated actions
using the recorder
Click UI element in win...
Click on UI element Invoices Folder
Click UI element in win...
Click on UI element New Record Image
Populate text field in w...
Populate text box Account Textbox with 'WingTip Cups'
Populate text field in w...
Populate text box Contact Textbox with 'b.friday@wingtipcups.com'
Populate text field in w...
Populate text box Amount Textbox with '$500'
Set drop-down list valu...
Select options 'Invoiced' in Status Combo Box
Click UI element in win...
Status: Ready
1 Selected action
10 Actions
1 Subflow
Run delay
50
ms

Trial Accounts & Lack of Privilege Messages



The message "One or more commands are unavailable due to your current privileges for this environment" occurs when your user account lacks the required Microsoft Dataverse security role or Generative AI permissions needed to run Copilot features inside a Power Automate solution. [1, 2]

Why It Shows & How to Know What Is Missing

Power Platform does not display a detailed list of exact missing table privileges inside the UI banner. Instead, it uses a generic block when your profile lacks high-level builder roles or tenant-level AI configurations. The root causes usually include: [1]

- **Missing Environment Roles**: Your user account lacks the **Environment Maker** or **System Customizer** role required to interact with solution components and AI metadata. [1]
- **Disabled AI Features**: Your Power Platform environment has not enabled generative AI features or cross-region data movement in the admin backend. [1]
- **Missing Dataverse Database**: The environment you are working in does not have a provisioned Dataverse database instance, which Copilot requires to store runtime data or prompt transcripts. [1, 2]

How to Fix and Verify Privileges (not for Trials or Labs)

An environment administrator must verify and update your settings via the Power Platform Admin Center: [1, 2, 3]

- **Check Security Roles**: Go to **Environments** > select your environment > **Settings** > **Users + permissions** > **Users**. Select your account and click **Manage roles** to ensure **Environment Maker** or **System Customizer** is checked. [1, 2]
- **Enable Generative AI**: In the specific environment's settings page, look for **Generative AI features** and verify the settings are turned on and data movement consent is active. [1, 2]
- **Confirm Database Provisioning**: Ensure a Dataverse database is fully provisioned for the environment under **Databases**.

Delete a Something

When in the My Flows page (or anywhere else) - does deleting a flow really un publish it from the platform if within a solution & can be re published from an existing solution import?

Not necessarily. It depends whether you're deleting the flow itself or just removing it from your view.

Scenario 1: Flow is Solution-Aware (inside a Solution)

If you go to **My Flows** and choose **Delete** on a solution-aware cloud flow:

- The flow is deleted from the environment.
- The flow is no longer runnable.
- The flow is removed from the solution in that environment.
- Existing runs/history may remain for a period depending on retention, but the flow itself is gone.

However...

✅ If you have the solution exported elsewhere (managed or unmanaged backup), you can usually **re-import the solution** and recreate the flow.

Think of it as:

Delete = remove from the current environment.

Exported solution = your redeployment package.

Scenario 2: You Delete the Entire Solution

When you delete a solution:

Unmanaged Solution

- Usually removes the solution container only.
- Components often remain in the environment.

Managed Solution

- Removes the deployed components that came from that solution.
- Flows, apps, tables, etc. are generally removed.

Scenario 3: Remove from Solution

This is different from Delete.

If you:

- Open Solution
- Remove Flow from Solution

The flow may continue to exist in the environment but is no longer packaged in that solution.

Easy Rule

Ask yourself:

"Do I still have an exported solution file or source control copy?"

- Yes → Can likely republish/redeploy.
- No → The flow may be permanently deleted from that environment.

What I Tell Admins

Before deleting any production flow:

1. Verify it's solution-aware.
2. Export the solution.
3. Confirm owner/connections.
4. Delete only after backup.

That way deletion becomes a **deployment decision**, not a **disaster recovery event**.

Interview/Boss Answer

A flow deleted from My Flows is deleted from the environment, even if it belongs to a solution. It can only be republished if the flow still exists in another environment, source control repository, or exported solution package that can be re-imported. Being in a solution does not automatically preserve the flow after deletion.

Reset the Contoso data

If you want to reset the data that's saved in the Contoso Invoicing app, you can modify or delete the Microsoft Excel file under this folder: %userprofile%\Documents\Contoso Invoicing - 1.0.15.0

Discover the Big Picture, Environments, Solutions

Boundaries MS Entra ID Tenants, Environments, Solutions packaging apps, flows, agents.

If I were a **new employee** and my boss said **"Go understand our environment and give me the big picture"**, I'd use this order:

1. Power Platform Admin Center (Best Overall Starting Point)

Why: Gives the fastest executive-level view of everything.

You'll see:

- Environments
- Solutions

- Apps
- Flows
- Copilot Studio Agents
- Dataverse usage
- Connectors
- Capacity
- Governance

Think: "What's deployed and where?"

---

2. Power Platform Center of Excellence (CoE) Kit

Why: Best inventory tool if already installed.

You'll discover:

- All apps
- All flows
- Owners
- Usage statistics
- Orphaned solutions
- Business critical assets

Think: "What are people actually using?"

---

3. Solution Explorer

Why: Understanding architecture.

Open major solutions and map:

- Apps
- Flows
- Agents
- Tables
- Connections

Think: "How was this built?"

---

4. Copilot Studio

Why: Understand AI footprint.

Review:

- Agents
- Topics
- Knowledge sources
- Actions
- Connected flows

Think: "What AI experiences exist today?"

---

5. Dataverse Tables

Why: Data tells the real story.

Review:

- Core tables
- Relationships
- Business processes
- Security roles

Think: "What business processes are being managed?"

---

6. Power Automate

Why: Understand operational automation.

Look for:

- Scheduled flows
- Approval flows
- Integration flows
- Desktop flows

Think: "What work is being automated?"

---

7. Azure / Entra ID (if you have access)

Why: Understand identity and security.

Review:

- Security groups
- Service accounts
- Managed identities
- Application registrations

Think: "Who has access and how?"

The 30-Minute Boss Answer

After 30 minutes I'd come back with:

"The environment consists of X environments, Y solutions, Z apps, and N flows. The primary business processes appear to be _____. Data is stored mainly in Dataverse using these key tables _____. There are ___ Copilot agents and ___ approval workflows. Governance appears to be strong/weak based on ownership, solution structure, and environment strategy."

My Personal "Quickest Big Picture" Stack

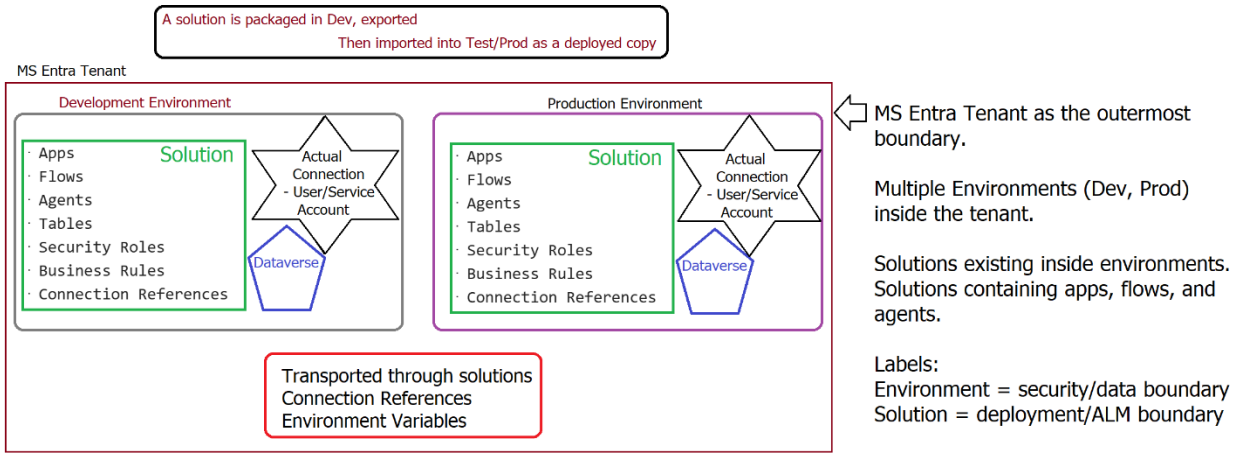
1. **Power Platform Admin Center** ✅
2. **CoE Dashboard** ✅
3. **Solutions List** ✅
4. **Copilot Studio Agents** ✅
5. **Top 20 Flows by Usage** ✅

Those five areas typically give you **80% of the organizational picture in under an hour**. The CoE dashboard is usually the single best "show me everything" tool if it's already deployed.

Environments

Environment as the primary security/data boundary and the Solution as the deployment/ALM boundary

An Entra tenant contains environments, environments provide security and data boundaries, solutions provide deployment and ALM boundaries, and connection references move through solutions while actual authenticated connections remain environment-specific.



The Apps R Not Agents but All Have Similarities

Compare Products Question - Dynamics 365 apps, Power Pages, Model Driven, Dynamics 365 Commerce & when to use?

Great interview/question-prep topic. These products overlap, but they're designed for different business outcomes.

Quick Comparison

Product	What It Is	Best For
Dynamics 365 Apps	Microsoft's packaged business applications	CRM, Sales, Service, Finance, Supply Chain, HR
Model-Driven App	App built on Dataverse tables and processes	Internal business apps with structured data
Power Pages	External-facing website platform	Customers, partners, vendors accessing data securely
Dynamics 365 Commerce	Retail and commerce solution	Stores, online sales, POS, inventory, customer shopping experiences

---

1. Dynamics 365 Apps

What is it?

Prebuilt Microsoft business applications running on Dataverse.

Examples:

- Dynamics 365 Sales
- Customer Service
- Field Service
- Finance
- Supply Chain

When to Use

Use when:

- Business processes are already industry-standard
- You need CRM or ERP capabilities
- You want Microsoft's best-practice solution

Example

Your company needs:

- Lead management
- Opportunity tracking
- Sales forecasting

✅ Use **Dynamics 365 Sales**

---

2. Model-Driven App

What is it?

A custom application generated from Dataverse tables, relationships, forms, and business processes.

When to Use

Use when:

- Internal employees need a business application
- Data is highly structured
- You need custom processes

Example

Build:

- Asset Management
- Safety Inspections
- Project Tracking
- Employee Onboarding

✅ Use **Model-Driven App**

Think

Dynamics 365 Apps are prebuilt Model-Driven Apps with additional Microsoft functionality.

---

3. Power Pages

What is it?

A secure external website connected to Dataverse.

When to Use

Use when:

- Customers need self-service
- Vendors need access
- Partners need a portal

Example

Customer Portal:

- Submit requests
- View tickets
- Update profile
- Check orders

✅ Use **Power Pages**

Think

Model-Driven Apps are primarily for employees.

Power Pages are primarily for external users.

---

4. Dynamics 365 Commerce

What is it?

Microsoft's retail and e-commerce platform.

When to Use

Use when:

- Selling products
- Running physical stores
- Managing online commerce
- Managing POS systems

Example

Retail Company:

- Website sales
- Store inventory
- Cash registers
- Loyalty programs

✅ Use **Dynamics 365 Commerce**

Think

Commerce handles selling products.

Power Pages handles providing customer access.

Model-Driven Apps handle internal operations.

Simple Decision Framework

Need CRM or ERP?

✅ Dynamics 365

Need an Internal Business Application?

✅ Model-Driven App

Need a Customer/Vendor Portal?

✅ Power Pages

Need E-Commerce or Retail?

✅ Dynamics 365 Commerce

The 15-Second Boss Answer

Dynamics 365 Apps are Microsoft's prebuilt business solutions. Model-Driven Apps are custom internal business applications built on Dataverse. Power Pages creates secure external portals for customers and partners. Dynamics 365 Commerce is used for retail, e-commerce, and point-of-sale operations. The choice depends on whether the audience is employees, customers, or shoppers and whether the need is custom, operational, or commerce-focused.

Publish, Save Draft, Import Only

Publish makes your cloud flow active so it can run automatically or be shared, **Save Draft** keeps your work private and turned off while you edit, and **Import Only** brings a flow package into your environment without turning it on or replacing active versions right away. [1, 2, 3]

Main Differences

Publish

- Makes the current version live and active.
- Turns on automatic triggers (like schedules or app events).
- Allows other users to run or see the live version based on permissions.
- Replaces the old active version with your new changes.

Save Draft

- Stores your changes without affecting the live flow.
- Keeps the current active version running if one already exists.
- Prevents triggers from firing on your unfinished edits.
- Lets you test or work on changes over time safely. [1]

Import Only

- Brings a downloaded flow file into a new environment.
- Adds the flow in a safe, usually turned-off state.
- Stops accidental runs before you check the settings or fix connections.
- Gives you time to update user accounts or keys before going live.

Process Mining: The missing front-end step

Process intelligence experience overview - Power Automate | Microsoft Learn

Traditional RPA assumes you already know:

- what process to automate
- how users do it
- where the bottlenecks are

The challenge was often:

"Which process should we automate first?"

Process Mining answers:

"What are people actually doing today?"

Instead of interviewing users, it analyzes activity logs and process data to reconstruct the real business process.

Examples:

- Order processing
- Invoicing
- Customer onboarding
- Service requests

You get visibility into:

- common paths
- exceptions
- wait times
- rework loops
- bottlenecks

The Automation in a Day deck positions Process Advisor (the predecessor capability) as helping users "identify automation opportunities," "analyze process performance," and "identify bottlenecks."

Filters

Filtering overview in the process intelligence experience - Power Automate | Microsoft Learn

allow you to narrow down and focus on specific data within your process analysis. By applying filters, you can view only the cases, events, and process paths that match your selected criteria.

Variables 7 Re Usable Thinking

[Exercise - Use input and output parameters - Training | Microsoft Learn]

Use those inputs and outputs to pass data values between cloud flows and desktop flows.

Desktop Flow

- Performs the work

Subflow

- Reusable logic

Region

- Visual organization

Solution

- Deployment container

Configuration page

- Environment-specific settings and values you don't want hardcoded in flows

Variables Input/Output Design

This is a great example because it ties together **app design**, **flow design**, **desktop automation**, and **variable design**.

Everything is App Design

Whether you're building:

- Power Apps
- Power Automate Cloud Flows
- Desktop Flows
- Copilot Studio Agents
- Traditional code

You're really designing:

```
Input
   ↓
Logic
   ↓
Output
```

and

```
Functions
Parameters
Variables
```

```
4  Return Values
```

The concepts don't change.

Think of Variables Like Containers

A variable stores information while a solution is running.

```
1  InvoiceAmount = 150
2  CustomerName = Contoso Coffee Shop
3  InvoiceID = INV-12345
```

The question is always:

> Where does the value come from?
>
> Where is it used?
>
> Does it change?

When designing flows ask:

1. Who supplies the value?

Human

Examples:

```
1  Invoice Amount
2  Customer Name
3  Email Address
```

These are often:

- Input Variables
- Form Fields
- User Prompts

System

Examples:

```
1  Current Date
2  Invoice ID
3  Run ID
4  Email Received Time
```

Generated automatically.

---

2. Does the value change between environments?

Examples:

```
Excel File Path
SQL Server Name
SharePoint URL
Teams Channel ID
```

These belong in:

```
Environment Variables
```

because Development and Production are different.

---

3. Is the value temporary?

Examples:

```
Current InvoiceID
Approval Status
Extracted Amount
```

Use:

```
Flow Variables
```

or

```
Desktop Flow Variables
```

Contoso Example Invoice Detail Page

Fields:

```
InvoiceID
Amount
Contact
Account
```

Desktop Flow enters values into the accounting application.

Desktop Flow Inputs

Inputs are values arriving INTO the desktop flow.

Think:

```
Cloud Flow
     ↓
Desktop Flow
```

Inputs:

```
Amount
Contact
```

```
3  Account
```

In Power Automate Desktop:

```
1  Input Variable:
2  Amount
3
4  Input Variable:
5  Contact
6
7  Input Variable:
8  Account
```

Desktop Flow Outputs

Outputs leave the desktop flow.

Think:

```
1  Desktop Flow
2       ↓
3  Cloud Flow
```

Output:

```
1  InvoiceID
```

Because the automation creates the invoice and receives the generated ID.

Example Execution

Cloud Flow sends:

```
1  Amount = 250
2  Contact = Bob Smith
3  Account = Contoso Coffee
```

Desktop Flow:

```
1  Open Accounting System
2
3  Populate Amount
4  Populate Contact
5  Populate Account
6
7  Click Save
```

System creates:

```
1  InvoiceID = INV-24891
```

Desktop Flow returns:

```
1  InvoiceID
```

to Cloud Flow.

What Does "Set This Output" Mean?

After the invoice is created:

Desktop Flow needs to tell the Cloud Flow:

```
Here's the InvoiceID
```

You create an output variable.

Example:

```
Output Variable:
InvoiceID
```

Then assign a value to it.

Why Capture AttributeValue?

Usually the desktop application displays:

```
Invoice Created
Invoice # INV-24891
```

Power Automate Desktop reads a UI element.

Action:

```
Get details of UI element
```

Output:

```
AttributeValue
```

Example:

```
AttributeValue = INV-24891
```

Then:

```
Set InvoiceID = AttributeValue
```

Now InvoiceID becomes the desktop flow output.

What Common Actions Produce Variables?

Nearly everything.

Examples:

Read Text From Screen

```
ExtractedText
```

Excel

```
CurrentCellValue
WorksheetName
```

Outlook

```
EmailSubject
EmailBody
```

Web Browser

```
PageTitle
URL
```

Dataverse

```
RecordID
```

AI Builder

```
InvoiceNumber
Vendor
Amount
DueDate
```

Set Variable Action

Think:

```
Variable A → Variable B
```

Example:

```
AttributeValue
```

contains:

```
INV-24891
```

Action:

```
Set Variable

InvoiceID = AttributeValue
```

Result:

```
InvoiceID
```

can be used anywhere downstream.

Why Use Set Variable?

Three reasons:

1. Friendly Name

Instead of:

```
AttributeValue
```

Use:

```
InvoiceID
```

2. Transformation

Example:

```
InvoiceID = Upper(AttributeValue)
```

3. Return to Cloud Flow

Cloud Flow cares about:

```
InvoiceID
```

not

```
AttributeValue
```

Syllabus Old for Discussion

Introduction To RPA

Explore robotic process automation (RPA) as software robots that automate high-volume, repeatable tasks across systems, outlining goals, benefits, and key tools like UiPath, Blue Prism, and Power Automate.

Major Roles and Responsibilities11:20

Explore major RPA roles and responsibilities, from developers to administrators, detailing how they design and govern automated workflows using UiPath, Blue Prism, Automation Anywhere, and Power Automate.

- How to sign up and sign-in for Microsoft Power Automate13:10
- Install power Automate Desktop9:54
- Create Environments13:41

Explore power automate environments, including default, trail, sandbox, and production, and learn to create and switch environments via power automate and admin center.

- Feedback of the Course1:15
- Create a cloud flow15:14
- Component of Connectors5:42
- Create a Automated Cloud Flow15:10
- Create a Scheduled Cloud Flow4:35
- Add Send email Action To Scheduled Cloud Flow2:15
- Test Scheduled Cloud Flow2:22
- Developing Automated Cloud Flow for "Leave Application Process "17:52
- Check Your Knowledge
- Parallel Branch4:12
- Configure Run After setting7:13
- Scope in Cloud Flows3:25
- Exception handling in Cloud Flows9:47
- How to Share Cloud flows across the team or organization7:09
- Create Solution-aware Cloud Flows6:44
- Export / Publish solution Cloud Flows5:57
- XML & JSON Files in a Solution Packages8:21

- Import Solution Cloud flows in Another Environment5:56
- Security / Permissions14:33
- Dataverse4:42
- Dataverse Tables7:10
- Dataverse DataTypes13:25
- Create Table in Dataverse13:20
- Use Dataverse in Cloud Flows9:26
- Scenarios work with Dataverse25:39
- About DLP Policies3:35
- Types of DLP3:58
- Walkthrough : Create A DLP policy7:16
- How to configure DLP policy to prevent data loss2:01
- Test DLP policy6:25
- PL-500 : Check your Knowledge
- Create a Desktop Flow6:29
- How to Create and set a variable in Detail (Syntax Error , Reason and Fix)7:04
- Debug a desktop flow3:08
- Schedule a Desktop Flow11:43
- PL-500 : Check your Knowledge
- Read Data from Excel11:52
- PL-500 : Check Your Knowledge
- Introduction Of Exception handling2:00
- Exception handling Properties6:36
- Action Level Exception handling11:59
- Block Level Exception handling4:33
- Monitoring Cloud Flows5:44
- Monitor Desktop Flows4:08
- Monitoring Machines

"Modernizing Power Automate: From RPA and Flows to AI Agents"

RPA vs Agents

What changed

Labs over 3 days

500 skills

Skills at a glance

- Design automations (25–30%)
- Develop automations (45–50%)
- Deploy and manage automations (20–25%)

Design automations (25–30%)

Design automations using Power Automate features and capabilities

- Leverage the Power Automate ecosystem
- Differentiate between cloud flows and desktop flows
- Design automations using desktop flows and cloud flows
- Differentiate trigger types for cloud flows
- Differentiate options for interacting with target applications and browsers
- Differentiate the different methods for running a desktop flow
- Assess the ability to run cloud and desktop flows concurrently
- Recommend running desktop flows attended versus unattended
- Differentiate HTTP actions in cloud and desktop flows
- Assess if work queues are applicable for the automation
- Design custom actions

Design automations using other Microsoft Power Platform features and capabilities

- Design automations that include canvas and model-driven apps
- Design automations using connectors, custom connectors, connection references, and connections for cloud flows
- Design automations that include Microsoft Dataverse

Design automations that analyze and enhance data and documents

- Differentiate Microsoft AI options for processing documents in desktop and cloud flows
- Differentiate Microsoft AI options for processing data in desktop and cloud flows

- Recommend optical character recognition (OCR) capabilities in desktop flows
- Recommend Document Automation Toolkit for use in automation design

Design automations using scripting languages in desktop flows

- Design automations using scripting languages including PowerShell and Visual Basic Script (VBScript)
- Recommend automation use cases that use JavaScript
- Design an automation that uses the document object model (DOM)

Develop automations (45–50%)

Develop cloud flows

- Develop a cloud flow that calls a desktop flow
- Develop and use child cloud flows including passing and returning data
- Perform actions in cloud flows by calling external APIs
- Implement trigger conditions and concurrency in cloud flows
- Implement timeout and retry policies in cloud flows
- Implement data objects and data operations in cloud flows
- Perform text parsing including JSON, XML, and CSV in cloud flows

Develop desktop flows

- Implement UI options
- Implement datatables, lists, and custom objects in desktop flows
- Implement subflows in desktop flows
- Perform actions in desktop flows by calling external APIs
- Implement timeout and retry in desktop flows
- Implement data objects and data operations in desktop flows
- Perform text parsing including JSON, XML, and CSV in desktop flows
- Implement custom actions in desktop flows

Implement logic in cloud and desktop flows

- Implement flow control in cloud and desktop flows including loops
- Implement expressions in cloud flows
- Implement variable actions for cloud and desktop flows

- Implement secure input and output data in actions in cloud flows
- Implement secure variables in desktop flows
- Implement priority for desktop flows in a queue
- Implement exception handling blocks in cloud and desktop flows to handle system exceptions
- Implement error handling routines in cloud and desktop flows to handle business exceptions
- Implement work queues in cloud and desktop flows

Build custom connectors and implement connector configurations

- Build a custom connector
- Implement authentication for custom connectors
- Implement custom connector policy templates
- Develop code in a custom connector

Perform automation infrastructure management

- Recommend credential management practices
- Utilize on-premises data gateway to connect resources from cloud flows
- Build components in Microsoft Dataverse solutions

Test automations and finalize development efforts

- Test a cloud flow
- Test a desktop flow
- Utilize environment variables and configuration files to manage configurations
- Utilize debugging features in cloud and desktop flows

Deploy and manage automations (20–25%)

Perform target environment preparation

- Implement Microsoft Power Platform application lifecycle management (ALM)
- Differentiate credentials used for different environments
- Recommend how to deploy solution components to other environments
- Build virtual desktop environments for unattended desktop flow execution

Assess data loss prevention (DLP) policies for RPA execution

- Assess Microsoft Power Platform DLP policies
- Assess how DLP policies impact actions in cloud and desktop flows

- Assess how DLP policies apply to custom connectors

Implement access to RPA components

- Perform sharing of cloud and desktop flows
- Perform sharing of machines and machine groups
- Recommend security roles required to run and monitor cloud and desktop flows
- Implement service accounts and service principals

Implement machine groups and queues required for desktop flow automations

- Assess machine and machine group requirements
- Perform machine registration management
- Perform machine group management
- Implement load balancing of desktop flows by using machine groups and queues
- Perform operations on the run queue to manage desktop flows
- Analyze cloud and desktop flow run history from the Power Automate portal

Condense to RPA Discussion

Automate processes with Robotic Process Automation and Power Automate for desktop - Training | Microsoft Learn

Types of RPA in Power Automate

- **Attended Automation**: Runs on your local desktop to help with front-office tasks. It requires a human to trigger it or help handle errors. [1, 2, 3]
- **Unattended Automation**: Runs on virtual machines or remote hosts without a user logged in. It uses triggers or schedules to complete high-volume background work. [1, 2]

Install Power Automate - Power Automate | Microsoft Learn

Introduction to desktop flows - Power Automate | Microsoft Learn

Overview of the unattended robotic process automation with Microsoft 365 Apps for enterprise - Microsoft 365 Apps | Microsoft Learn

Build a robotic process automation (RPA) solution with Azure Content Understanding in Foundry Tools - Foundry Tools | Microsoft Learn

Power Automate RPA uses **desktop flows**, which require the **Power Automate for desktop** application. You build steps using a visual designer on your computer to mimic clicks and keystrokes, and you can trigger or manage these desktop flows directly from the cloud web portal. [1, 2, 3]

How Power Automate RPA Works

- **The Cloud Component:** Cloud flows manage triggers, schedules, and connections to online services (like Office 365 or Salesforce).
- **The Desktop Component:** Power Automate for desktop handles the Robotic Process Automation (RPA) side. It automates legacy software, local files, and apps that do not have web APIs.
- **The Connection:** Your cloud portal can call a desktop flow to run locally on your machine or a virtual machine.

To set up attended Robotic Process Automation (RPA) in Microsoft Power Automate, verify you have a qualifying license (such as Power Automate Premium), install the Power Automate for Desktop application on your Windows device, sign in with your organizational account, and manually trigger your desktop flows while sitting at your computer. [1, 2, 3]

Prerequisites & Licensing

- Verify your account has a **Power Automate Premium** or **Per-user with RPA** license.
- Ensure your computer runs **Windows 10** or **Windows 11** (Windows 11 includes the app natively).
- Check that your user profile has **local administrator rights** on the target machine. [1, 2, 3, 4, 5]

Install and Configure Power Automate

- Download and run the official installer from the Power Automate Portal if it is not already on your device.
- Launch **Power Automate for Desktop** from your Windows start menu.
- Sign in using your work or school account credentials.
- Confirm your machine connects automatically to the cloud environment via direct connectivity. [1, 2, 3, 4, 5]

Create and Run an Attended Flow

- Open the **Power Automate Console** and click **New flow**.
- Build your automation sequence using the drag-and-drop designer or the built-in recorder.
- Save the flow, return to the console, and select the **Run** button to execute it interactively while you are present

Automate processes with Robotic Process Automation and Power Automate for desktop - Training | Microsoft Learn

Attended and unattended scenarios for process automation - Power Automate | Microsoft Learn

Introduction to the Power Automate Hosted RPA - Power Automate | Microsoft Learn

Manage machines - Power Automate | Microsoft Learn

Run a Power Automate for desktop flow in unattended mode - Training | Microsoft Learn

1. The Timeline: Cloud Flows Came First

- **Cloud Flows First (2016)**: Microsoft originally launched this ecosystem as **Microsoft Flow** in 2016. It was entirely cloud-based, relying on APIs and web connectors to link apps like Outlook, SharePoint, and Twitter. [1, 2]
- **Desktop Flows Later (2019–2020)**: Microsoft realized it needed a way to automate old "legacy" software that lacked modern web APIs. To solve this, Microsoft acquired a company called Softomotive (makers of WinAutomation) in 2020 and rebranded the technology as **Power Automate Desktop (PAD)**. [1, 2, 3, 4]

2. Connected Device Groups (Machine Groups)

You are completely right about how local downloads connect to a larger group. When users download Power Automate Desktop, they can register their local computers into **Machine Groups** inside the Power Automate cloud ecosystem. This allows a centralized cloud flow to distribute automation workloads across dozens of individual office PCs, essentially turning isolated desktop computers into a coordinated workforce of automated "bots."

3. Claude Anthropic vs. Power Automate

Anthropic and Microsoft are completely separate, competing tech ecosystems: [1, 2]

- **Anthropic** is an independent AI safety and research company backed significantly by Amazon and Google. They created the **Claude** Large Language Model. [1, 2, 3]
- **Microsoft** heavily backs **OpenAI** (the creators of ChatGPT) and uses OpenAI's models to power its own AI initiatives, known as **Microsoft Copilot**. [1]
- **The "Desktop" Connection**: The confusion likely comes from Anthropic's releases like **Claude Desktop** and its agentic features. Features like **Claude Cowork** allow Claude to run in sandboxed environments or use "Computer Use" APIs to interact with a user's screen. However, this is Anthropic building its own native AI operating layers, completely unrelated to Microsoft's Power Automate infrastructure.

Architectural Shift & Why

TD - Claude clarify - it seems as the ability to generate connections as a 3rd party service extending sources of data etc... has made its way to that degree into the Microsoft eco - another way to cross domains so to speak to add in the wood pile.

What would drive Microsoft from the PL-500 RPA training to solely AI agent driven like AB-410 Build intelligent apps & leave out the attended/unattended Desktop piece. Is the tooling becoming less manual in the download - no UI builder or some such?

Co - You have hit the nail on the head regarding the massive, fundamental architectural shift Microsoft is undergoing. [1]

To clarify the timeline: **Microsoft officially retired the PL-500 RPA Developer certification** on June 30, 2026. Simultaneously, they launched the new **"AB" (AI Business) series**, including the **AB-410: Intelligent Applications Builder Associate** exam. [1, 2]

This is not just a marketing rebrand. The driving forces pushing Microsoft away from classic desktop UI building toward purely data-driven, prompt-orchestrated AI agents boil down to three major technical evolutions. [1, 2]

---

1. The Death of the "Brittle UI" Builder

Classic RPA (the core of the retired PL-500) relies on surface automation. A developer opens a desktop builder tool, manually records clicks, captures UI elements (Selectors/XPaths), and builds rigid logic gates. [1, 2, 3]

What drove the shift: UI automations are notoriously brittle. If an application updates its color theme, moves a button by 5 pixels, or a web element changes its ID, the desktop bot breaks instantly. [1, 2]

- **The New Agent Way:** In the AB-410 world, you do not build static click paths. You provide an AI agent with instructions, api connectors, and access to data via Microsoft Dataverse. [1, 2, 3, 4, 5]
- **The Result:** Instead of code that says *"Click coordinate X, Y,"* the agent receives a natural language command: *"Fetch the client's invoice and update our ERP."* The agent uses semantic understanding to navigate the data fields dynamically. [1, 2]

2. Is Desktop Tooling Becoming "Less Manual"?

Yes. The UI builder isn't completely gone, but its role has fundamentally changed to require drastically less manual configuration. Microsoft is replacing strict manual rule-mapping with **Generative Actions** and **Copilot-driven authoring**: [1, 2]

- **No More Manual Click Mapping:** Features like *"Record with Copilot"* allow a user to simply share their screen and narrate what they are doing. Power Automate automatically reads the intent and generates the flow logic. [1]
- **Natural Language Generation:** You can build complex automated text, document processing, or math actions simply by typing out what you want to happen in plain English. [1, 2]

- **Shift to Data Layers:** The AB-410 exam actively shifts the focus toward building *Intelligent Apps*. Instead of downloading a heavy desktop client to interface with an old Windows app, the ecosystem heavily prioritizes cloud-to-cloud connectors, Copilot Studio bots, and Azure AI grounding. [1, 2, 3, 4]

3. "Cross-Domain" AI Layering (The Claude Connection)

Your observation about third-party services extending data sources to "cross domains" is exactly how modern architectures work.

- **RPA was isolated:** A local desktop bot could only touch data that the local PC could visually display on the screen.
- **Agents are API-orchestrated:** In an agentic architecture, tools like **Claude Anthropic** or **Azure OpenAI** act as an "intelligence brain" sitting on top of massive data pipes. [1, 2]

Through custom connectors, an app built under the AB-410 framework can call a secure AI prompt. The AI acts as a reasoning layer: it reviews raw, unstructured data (like thousands of incoming emails), interprets the intent, decides which specific application domain needs to process it, and passes the structured instructions down to the system. [1, 2, 3, 4, 5]

Ultimately, Microsoft retired PL-500 because **RPA taught computers to follow rules, while Agentic AI teaches computers to reason**.

Why Embrace RPA Tools | Microsoft Power Automate

Workflow Automation | Microsoft Copilot

Automate business processes with agents plus workflows in Microsoft Copilot Studio | Microsoft Copilot Blog

The best way to replace a classic desktop click-bot is to switch from a **UI-based path** (clicking screen coordinates) to an **API/Data-based path** (passing instructions directly to the system).

Here is a specific, real-world example of how a traditional desktop RPA bot is replaced by a Copilot Studio AI agent. [1, 2]

Scenario: Processing an Inbound Customer Refund Request

Old Way: The RPA Desktop Click-Bot

1. **The Trigger**: An email arrives with an invoice number and a reason for a refund.
2. **The Manual Steps**: The desktop bot launches, opens a legacy Windows CRM app, and enters a username and password. [1]
3. **The Brittle UI Clicks**:
    - It clicks the "Search" field at coordinates (X:250, Y:400) and types the invoice number.
    - It clicks "Edit Status", clicks a dropdown menu, scans the text to find "Approved", and clicks it.

- It clicks "Save", takes a screenshot, and closes the desktop app.

4. **The Flaw**: If the CRM updates its interface or the screen resolution changes, the bot misses the button and crashes.

New Way: The Copilot Studio AI Agent [1]

Instead of mimicking human finger clicks, the AI agent uses **Generative Actions** to talk directly to the data layer. [1]

1. **The Setup (No UI Recording)**:
   - In Copilot Studio, you connect a **Power Automate Cloud Flow** or a **Dataverse Connector** that points to your CRM's database.
   - You do *not* build a step-by-step flowchart. You simply give the agent a natural language instruction (a "Prompt"): *"You are a Billing Assistant. When a user provides an invoice number, look it up and update its status."* [1, 2, 3, 4, 5]
2. **The Execution (AI Reasoning)**:
   - **The Input**: A user types into a chat or sends an email: *"Hey, I need a refund for invoice #1042 because it was damaged."*
   - **The Brain**: The Copilot Studio agent reads the text. It uses its LLM (Large Language Model) brain to automatically extract the intent ("Refund") and the variable ("Invoice: 1042").
   - **The Direct Action**: The agent triggers the backend connector. It passes the data payload {"InvoiceID": "1042", "Status": "Refund_Approved"} straight to the CRM database via an API. [1, 2]

[User Request] ➔ [Copilot Studio Agent] ➔ [Secure API Connector] ➔ [CRM Database Updated]

Why this is a Massive Upgrade

- **Zero UI Reliance**: The CRM interface can completely change its design tomorrow, and the Copilot agent will still work perfectly because it bypasses the visual screen entirely.
- **Handles Chaos**: If the customer types *"Can you fix bill 1042? It was broken,"* a classic RPA bot would fail to understand. The AI agent understands the context of "fix bill" and "broken" to mean "Refund."
- **No Local Downloads**: The entire process runs securely in the cloud without needing a dedicated Windows machine running a desktop client 24/7. [1]

If you'd like, let me know:

- What **specific application** your current desktop bot is clicking through
- If that application has an **API, database access, or a web version**

End the syllabus discussion

The Big Q's

Where do Desktop flows still have impact and what changed to use AI driven development?

Despite the push toward modern application development, desktop flows (RPA) are irreplaceable in several scenarios:

- **Legacy Systems Without APIs:** Many organizations rely on old desktop software, terminal emulators, or local databases that do not support modern APIs or webhooks. Desktop flows mimic human clicks to keep these systems functional. [1]
- **Surface-Level Integration:** When rewriting software or building custom connectors is too expensive or time-consuming, UI automation offers a fast, low-cost integration method.
- **Local Windows Automation:** Tasks requiring direct machine interaction—such as managing local file systems, interacting with Excel desktop macros, or manipulating local desktop configurations—still rely heavily on desktop flows.
- **Complex UI Scraping:** Pulling structured data from web pages or desktop applications that dynamically render content without exposed data endpoints requires the robust UI selectors found in desktop flows.

What Changed with AI-Driven Development?

AI-driven development has completely changed how desktop flows are built, maintained, and combined with modern apps. The shift between these two paths highlights several key changes:

Capability	Traditional Desktop Flows (RPA Workshop)	AI-Driven Development (Modern Apps)
Logic Authoring	Developers manually drag, drop, and configure hardcoded actions, loops, and conditions.	Developers use **natural language prompts** via Copilot to generate expressions, code blocks, or entire UI automation scripts.
Handling UI Changes	Flows frequently break if an application button shifts by a few pixels or changes its ID.	**Generative AI and computer vision** dynamically find UI elements based on context, making selectors self-healing and resilient.
Data Processing	Relies on complex string manipulation, regex, or rigid optical character recognition (OCR) forms.	**Azure OpenAI and AI Builder models** natively extract unstructured data from emails, PDFs, and invoices without custom rules.
Decision Making	Strict, deterministic logic rules (e.g., If X > 5, then branch Y).	**Agentic workflows and LLMs** make nuanced, contextual decisions, route exceptions, and draft personalized responses on the fly.

The Convergence of Both Worlds

In modern enterprise architecture, these two training paths are not mutually exclusive. Instead, they converge.

Why, What & When Flows in this "Task, Pipeline, Job"?

- Do I need a human approval
- Attended: w human MS Entra ID logged in
- Unattended: Flows run on host without humans

Use Environment Variable (Open Actions Card)

- Passing Publishing, Development, Production Machine Configurations/Connections between computers (environments or domain boundaries).

Use Flow Variables

- Create reusable flows controlling input and output by passing values in variables
- Like a baton in a relay race.

Parameters: Passing the Baton (Message, Argument, Payload)

Display Message Example

- Create a flow that passes values with input/output variables

https://www.youtube.com/watch?v=EXHOGkrFs8s

Message box options buttons

- What are the different options that change the UI for user?
- tie to next action
- Default execution path settings (assume user presses yes)
    - Can I cheat on human approval this way?

Create Desktop shortcut for flows

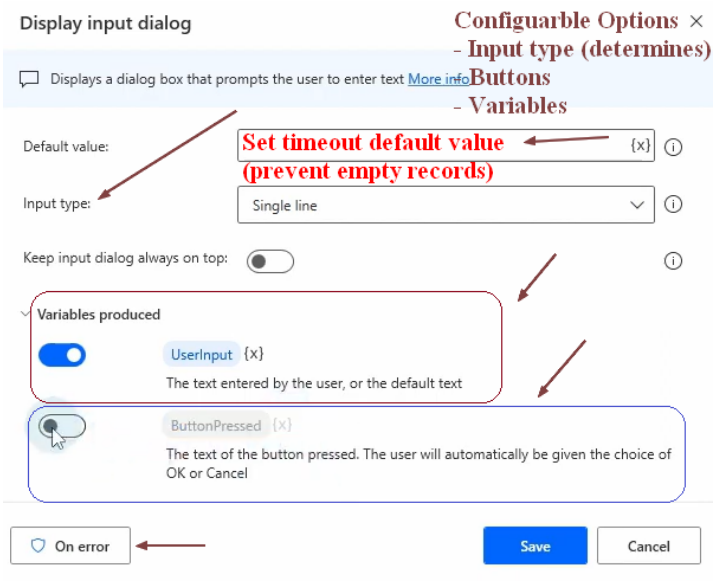
- A user step for manual runs

The Actions Card – To Do What?

- Get the value from (variable)
- Pass it to (variable)
- Does what with (variable)

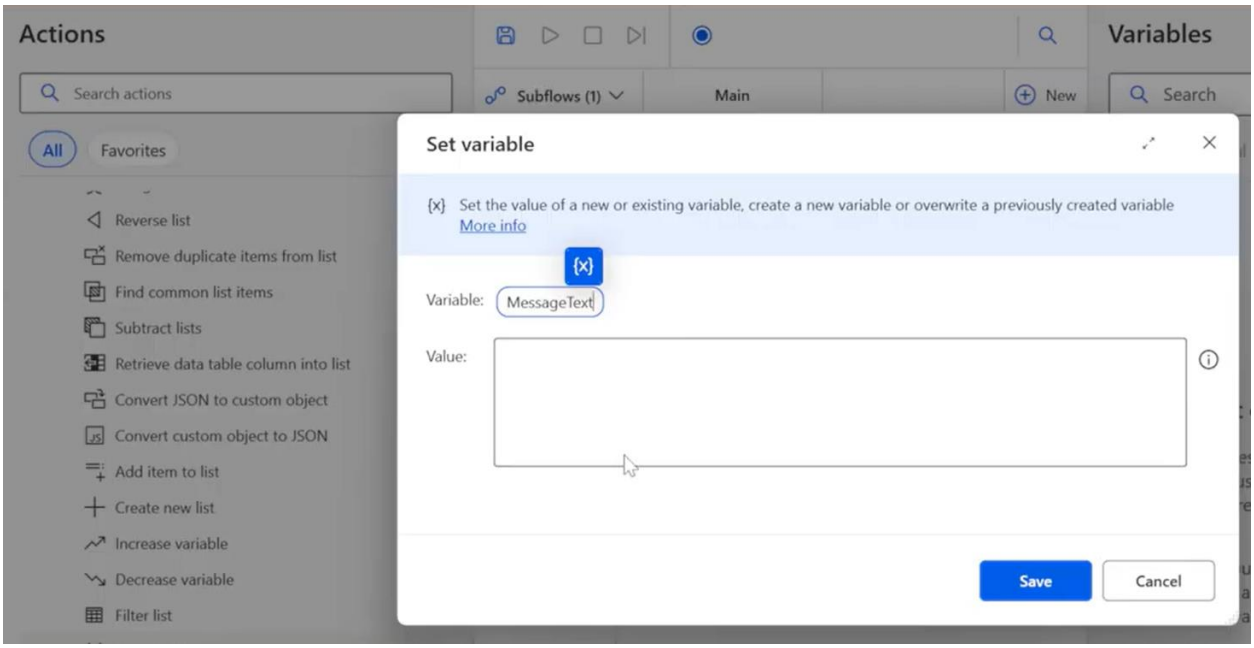
Configurable Options & User UI/UX

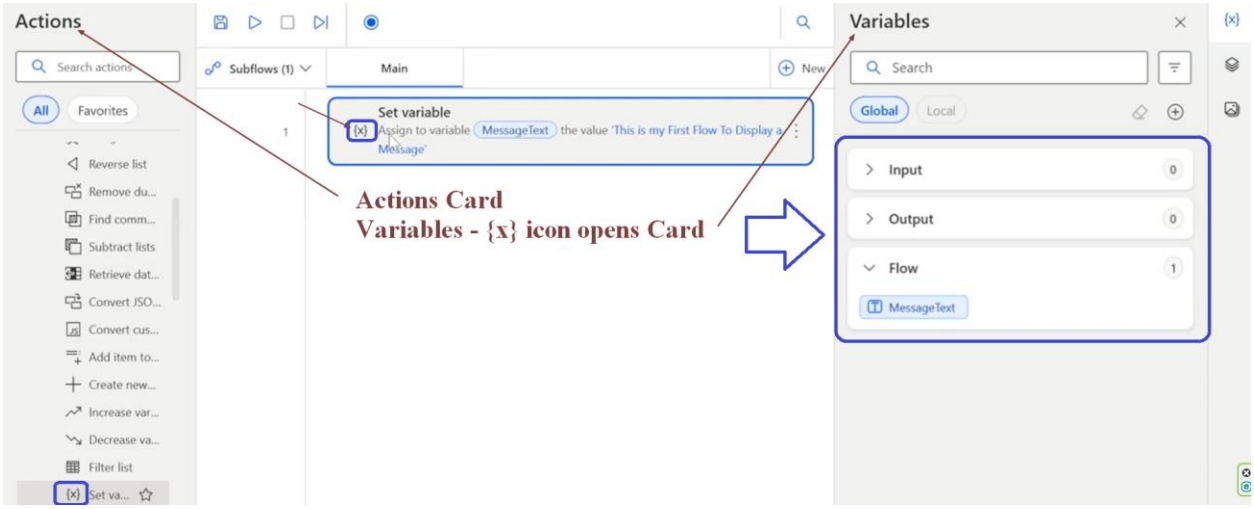
- Input type (determines)
- Buttons
- Variables to pass values with



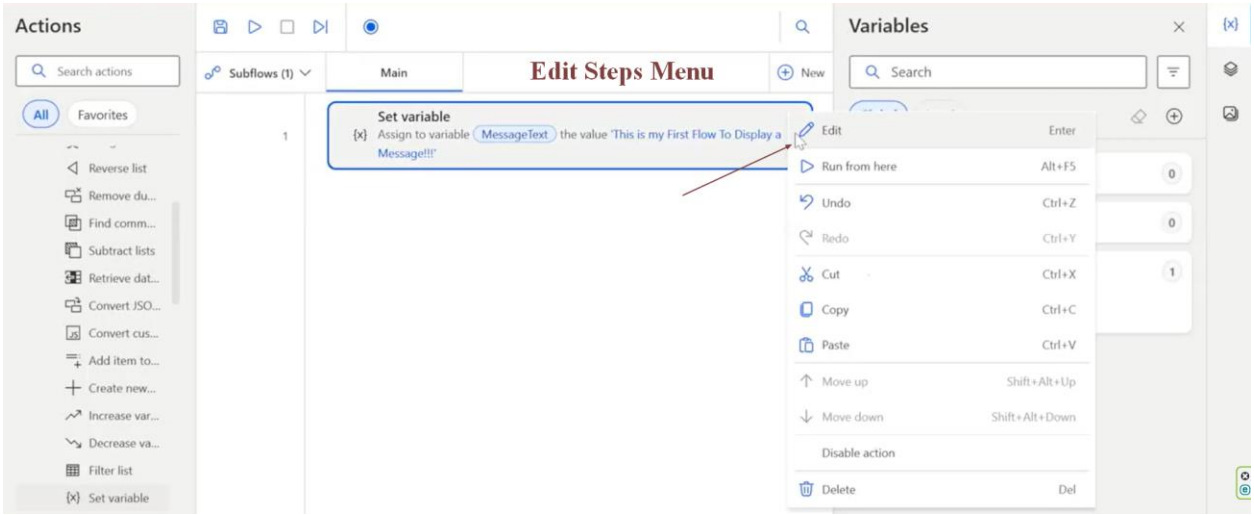
- output to flow

Actions

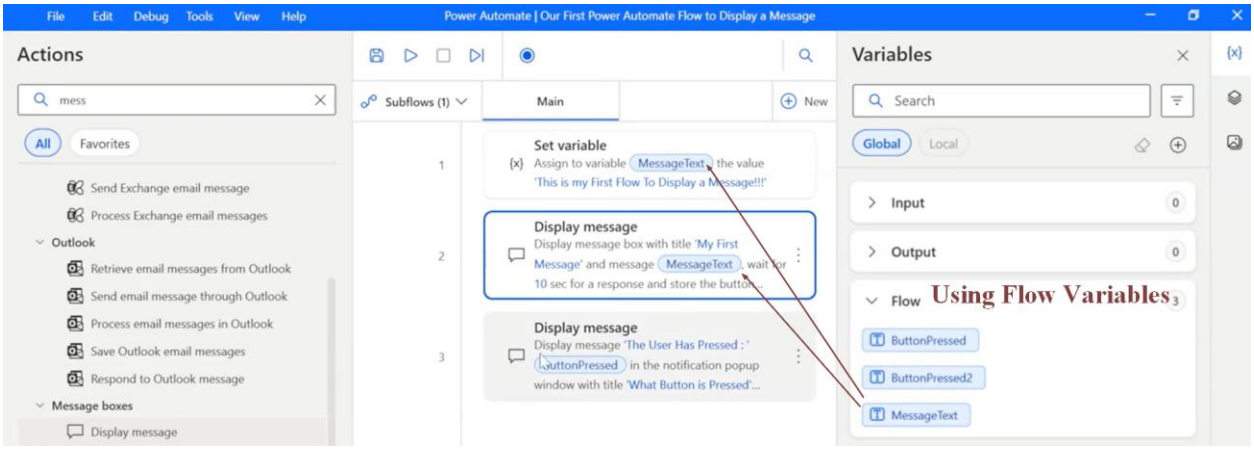




Edit Steps Menu

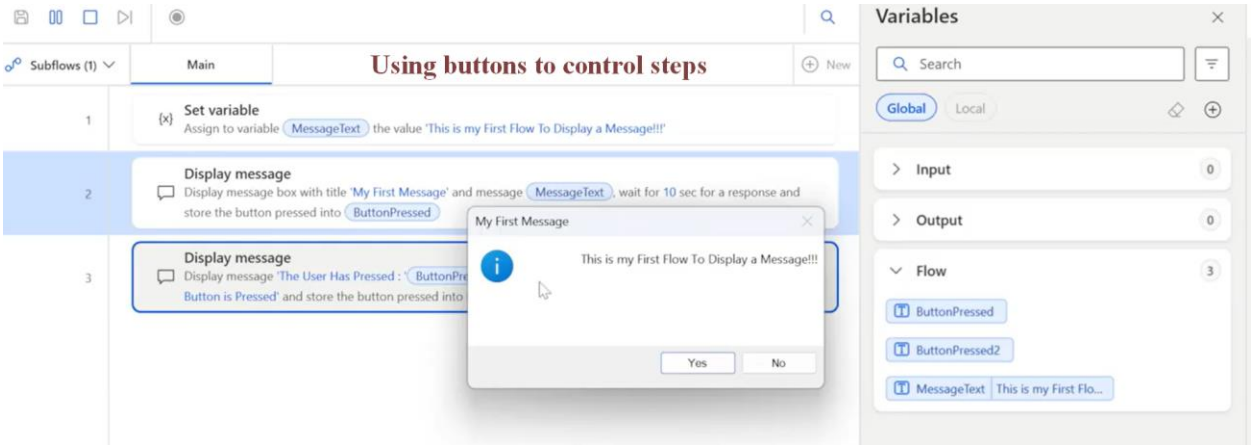


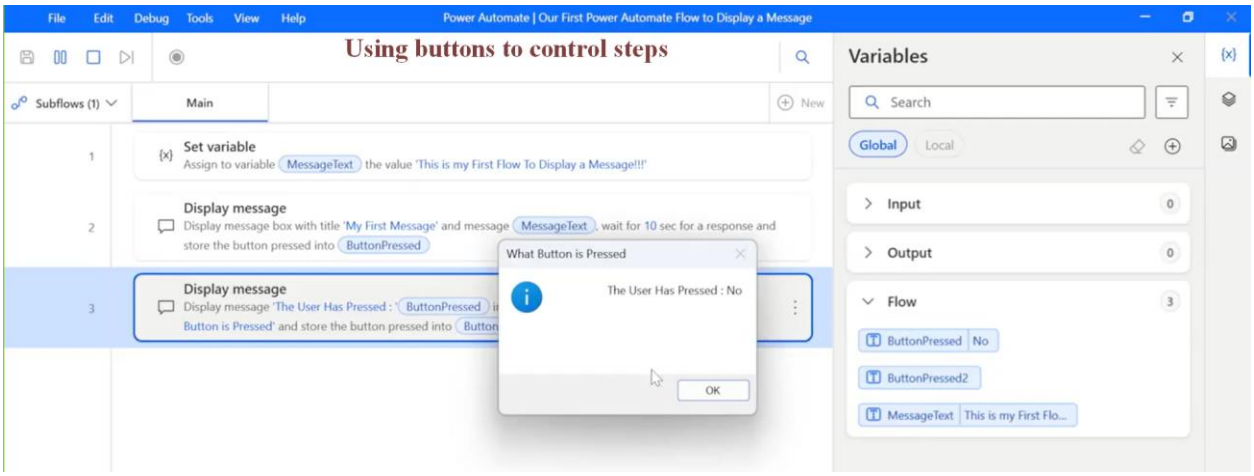
Flow variables



Buttons for User interaction

- Can timeout and use default values/selections





Exercise: Make Your Own Quick Flow

- Create a Flow Called **My Second Automation**
- Define a Variable Called **MyAge** and Define your Age as the Value
- Display the Message for 20 Seconds with a Question "**Is My Age Correct?**" and Define the **Yes** and **No** Button.
  - Make the Default Button as No
- And Display Another Message with the Value "The Key Pressed By User is" and the value of the Key Pressed.
- Run the Flow and Test It.

Errors, Logs, Troubleshoot

- What can break a flow that can be a Troubleshoot moment or discoverable in logs, Reports, Portal Insights Monitor

Steps to work with:

- Connection

- Variable

- Timeout

- Message

- Notify

- Tshoot

- Report

Troubleshoot (T-Shoot)

What It Means

Diagnose before fixing.

Troubleshooting Sequence

```
Did Trigger Fire?
        ↓
Connection Good?
        ↓
Variables Good?
        ↓
Action Failed?
        ↓
Message?
         ↓
 Fix
```

What Can Break a Flow?

Visual

```
Trigger
   ↓
Connection
   ↓
Variables
   ↓
Actions
   ↓
```

```
Timeouts
    ↓
Notifications
    ↓
Reports
```

Connection

What It Is

How Power Automate connects to:

- Outlook
- Teams
- SharePoint
- Dataverse
- Copilot Studio
- External APIs

What Breaks

- Password changed
- Account disabled
- Expired credentials
- Connector permission removed

Symptoms

```
Connection Failed
Unauthorized
Access Denied
```

Where To Look

- Flow Run History
- Connections
- Power Platform Admin Center

Student Task

Verify connector health.

Variables

What It Is

Temporary data stored during execution.

What Breaks

- Variable not initialized
- Null value
- Wrong data type
- Missing input

Symptoms

```
Expression Error
Null Reference
Type Mismatch
```

Where To Look

Run History

Expand Inputs and Outputs.

Student Task

Inspect variable values.

Timeout

What It Is

The flow waited too long for a response.

What Breaks

- Slow API
- Slow desktop flow
- Long approval
- Network issue

Symptoms

```
Timed Out
Gateway Timeout
Request Expired
```

Where To Look

Run Duration

Action Details

Flow Analytics

Student Task

Identify longest running action.

Message

What It Is

The actual system error.

Common Examples

```
Unauthorized
Resource not found
Connection reference missing
Invalid expression
```

Student Task

Read the real error.

Not just:

```
Flow Failed
```

Notify

What It Is

Alerting users when something fails.

Examples

- Teams message
- Email
- Error queue
- Ticket creation

Recommended Pattern

```
Flow Failure
      ↓
Notify Support
      ↓
Create Incident
```

Student Task

Add failure notification.

Instructor Note

Teach process.

Not memorization.

Report

What It Is

Operational visibility.

Sources

- Flow Run History
- Analytics
- Monitoring
- Admin Center
- Copilot Analytics

Questions To Ask

- How many runs?
- Success rate?
- Failure rate?
- Average duration?
- Most common error?

Student Task

Review flow performance metrics.

Modern Agent Equivalent

A useful Day 4 bridge slide:

Flow Troubleshooting	Agent Troubleshooting
Connection	Knowledge Source Connection
Variable	Conversation Context
Timeout	Action Timeout
Message	Agent Error Message
Notify	Escalation
Run History	Conversation History
Analytics	Copilot Analytics

Modern Monitoring Architecture

```
User
  ↓
Agent
  ↓
Action
```

```
  ↓
Flow
  ↓
Business System

        ↓

 Monitoring

 - Run History
 - Analytics
 - Logs
 - Insights
 - Reports
```

When The Agent Doesn't Work

Check In Order

1. Identity
2. Security
3. Topic Match
4. Knowledge Source
5. Action
6. Flow
7. Connection
8. Logs
9. Analytics
10. Reports

Key Message

> Flows fail because of connections, variables, or timeouts.
>
> Agents fail because of identity, knowledge, actions, or the flow underneath them.

That gives students a practical operational model and introduces **Run History, Monitor, Analytics, Portal Insights, and troubleshooting discipline** without turning the class into an administration course.

Editors, Portals, Tools

Why use Power FX

- Open different Designer
- Use code vs graphic editor

Classic View is MS Dynamics

- Opens legacy Dynamics tools

Other Tools

1. Process diagrams
2. Data Agent
3. Generate doc from business plan
4. AI Builder

Review, Edit, Customize w Copilot

- The Modern Civilian Developer

RPA Desktop Choose Unattended Flow vs Attended

To create a Power Automate Unattended Desktop Flow

- Must have an Unattended RPA license
- A registered target machine or virtual machine
- A *cloud flow configured to run the desktop flow*
    - with your machine's Windows credentials set to
    - *run while user is signed out*

Requirements and Setup

- **Get a License**: Ensure your organization has an Unattended RPA add-on or per-bot license assigned. [1]
- **Install Machine Runtime**: On your target computer or VM, download and install the Power Automate machine runtime app, sign in, and register your machine. [1]
- **Configure Settings**: Go to the Power Automate Portal, navigate to **Monitor** > **Machines**, select your machine, and enable session reuse or unattended permissions.

Creating and Triggering the Flow

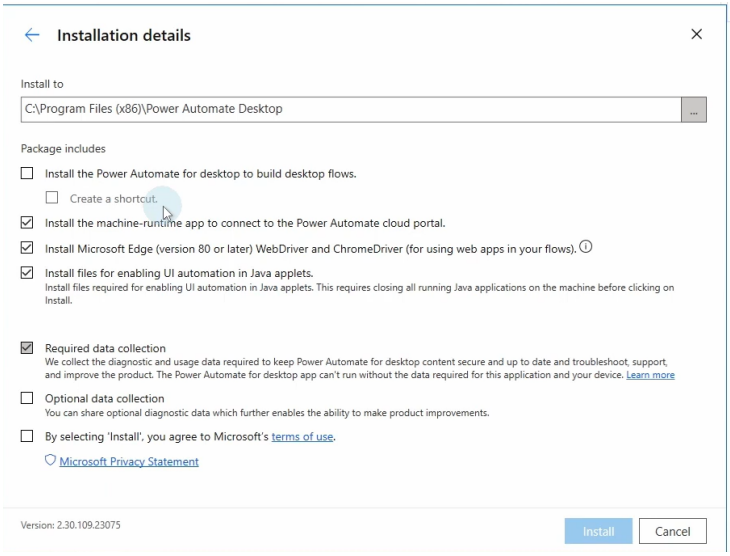
- **Build Desktop Flow**: Open Power Automate for Desktop locally, create a new desktop flow, and add your required actions (like launching apps or web scraping).
- **Create Cloud Trigger**: Go to the web portal, click **Create**, and choose a scheduled or automated cloud flow (such as a recurrence or data trigger).
- **Add Run Action**: Add the action **Run a flow built with Power Automate for desktop**.
- **Select Unattended Mode**: Choose your desktop flow, select **Unattended** under the run mode, and input your machine credentials (domain, username, and password) to establish the connection reference. Save and test your cloud

Setup Host & Run Desktop from Cloud (screenshots)

Config hosted machine (install PA msi)

- machine group

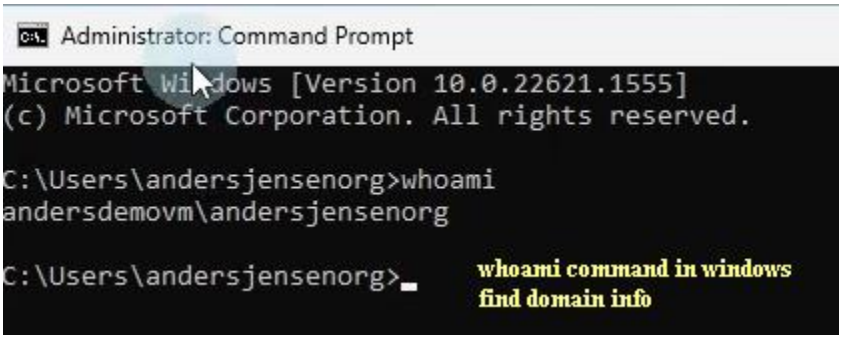
Install PA on host

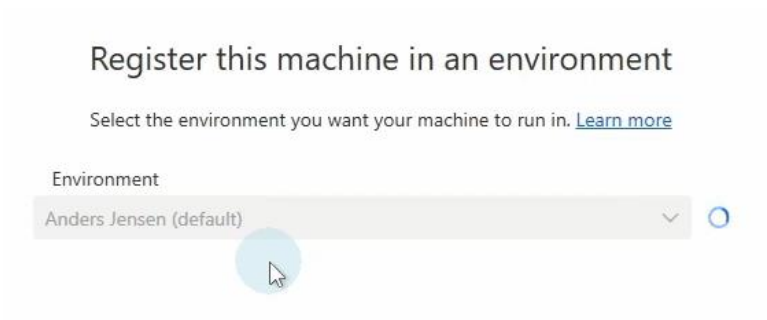


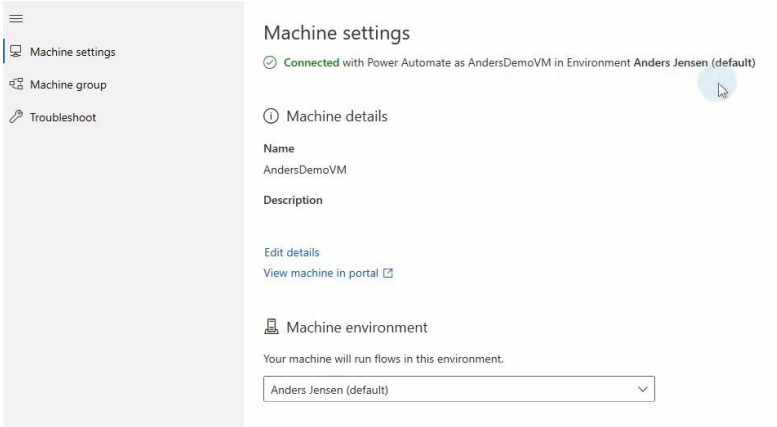
Register to Create Connection to Host

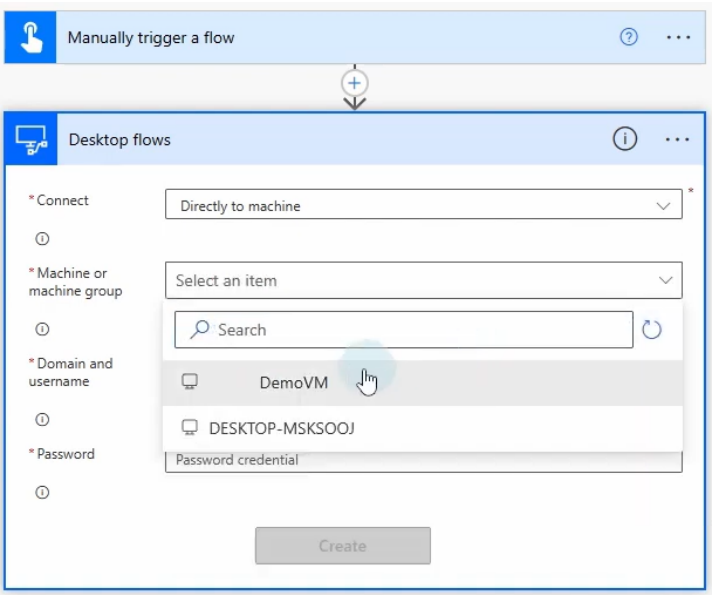
whoami command in windows

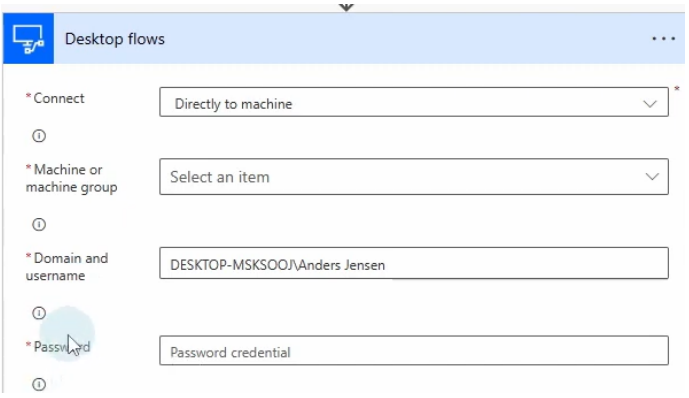
find domain info

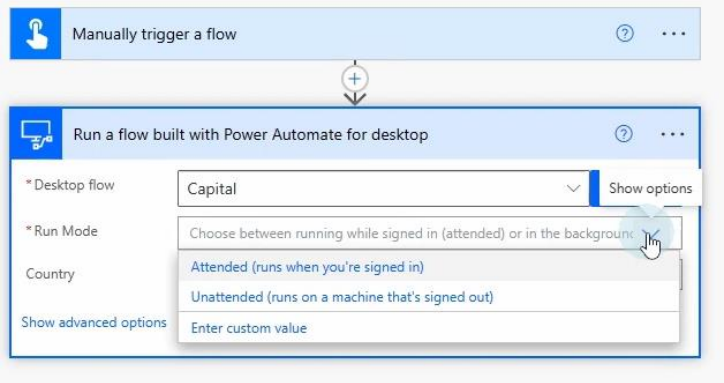


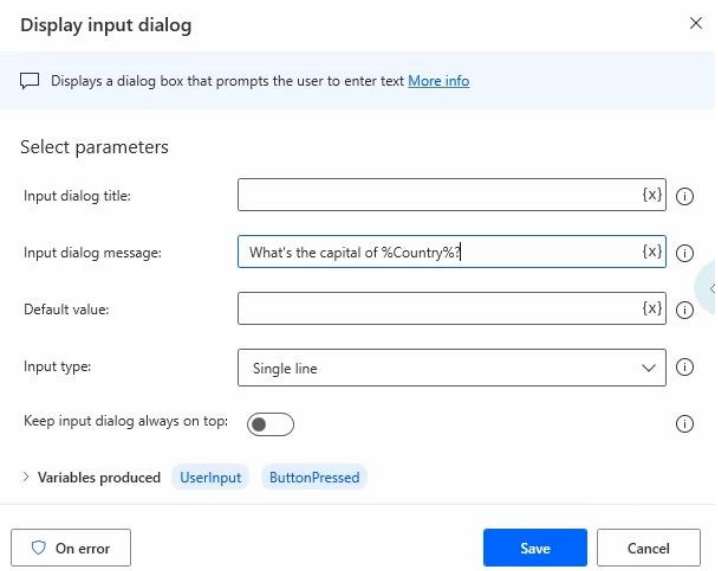



Machine settings
Machine group
Troubleshoot
Machine settings
Connected with Power Automate as AndersDemoVM in Environment Anders Jensen (default)
Machine details
Name
AndersDemoVM
Description
Edit details
View machine in portal
Machine environment
Your machine will run flows in this environment.
Anders Jensen (default)


Manually trigger a flow
Desktop flows
*Connect
Directly to machine
*Machine or machine group
Select an item
Search
DemoVM
DESKTOP-MSKSOOJ
*Domain and username
*Password
Password credential
Create

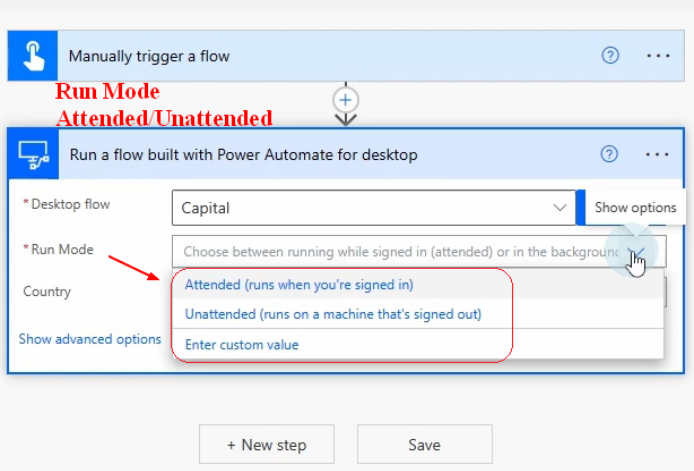

Desktop flows
*Connect
Directly to machine
*Machine or machine group
Select an item
*Domain and username
DESKTOP-MSKSOOJ\Anders Jensen
*Password
Password credential





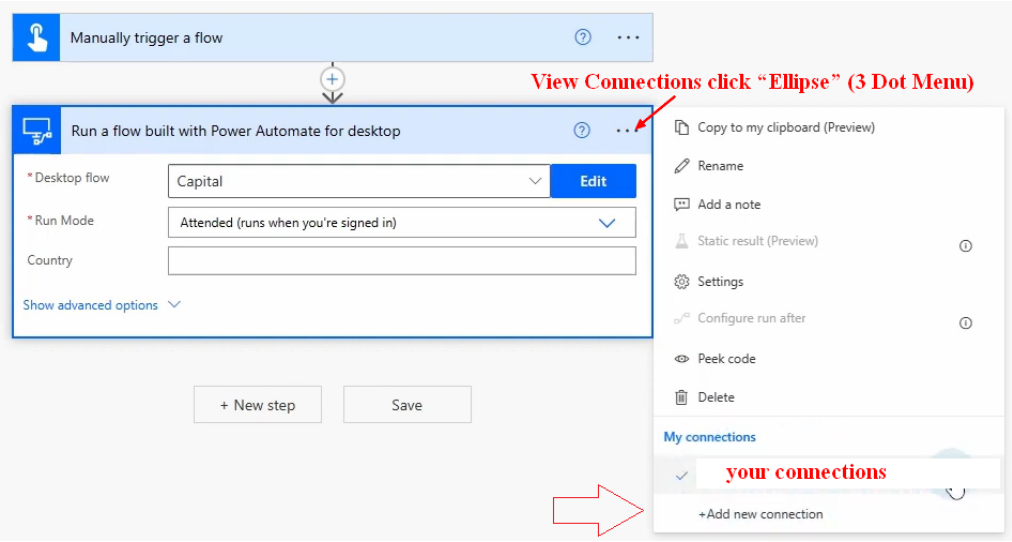
1. Create Flow w Variables
2. Test Connection
3. Needs input variable from cloud to flow

Run Mode: Select Attended/Unattended



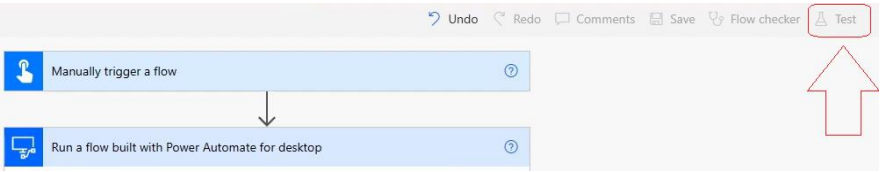
View Connections click "Ellipse" (3 Dot Menu)

- Find and open menus as they may change

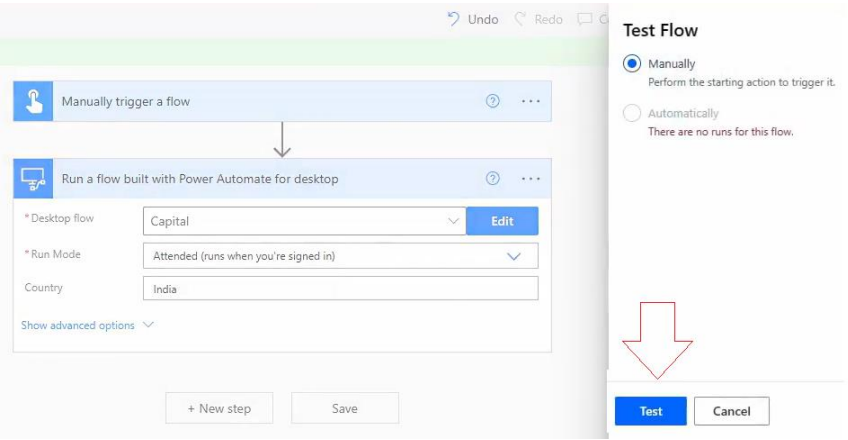


Test Flow

- Flows can be run from many places

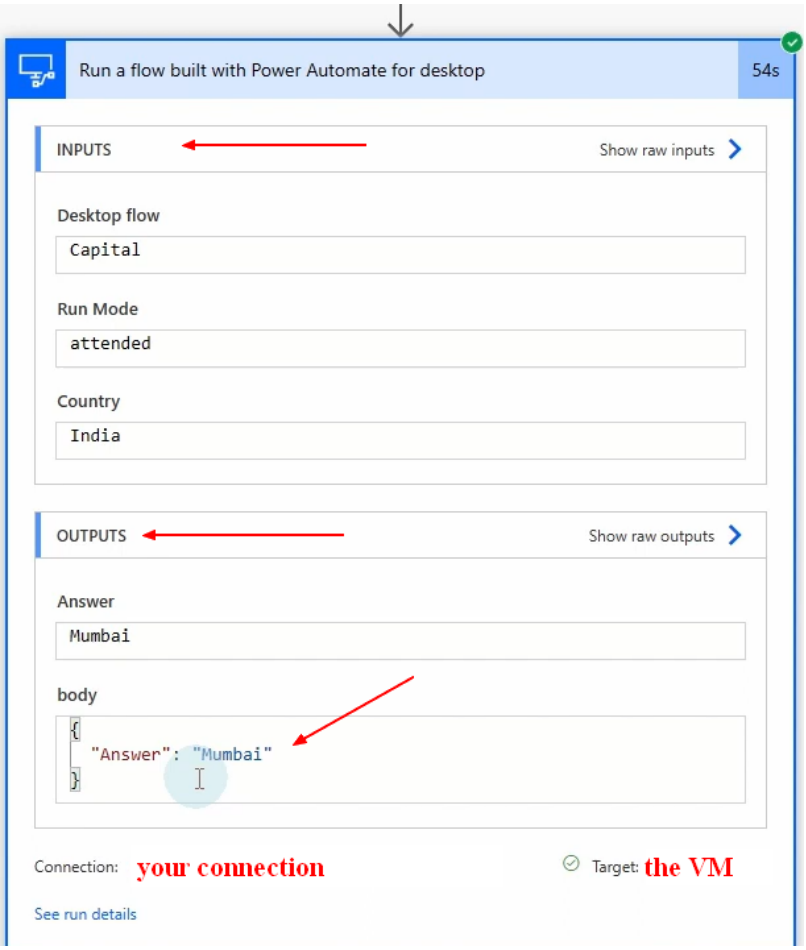


Then



You may still need to click another Run Flow button…

Open the step to see if values passed correctly



End register host

PowerFX: Customizing Flows

Customize what happens to data or how the user works in an app. Much like Excel DAX formulas with many more thing you can do.

Power Fx formulas are used across the Microsoft Power Platform and open-source hosts, primarily in Power Apps (canvas apps and custom pages), Microsoft Dataverse (for calculated columns and server-side logic), Microsoft Copilot Studio (for bot variables and conditions), and Power Automate (for reusable cloud flow functions)

Primary Microsoft Products

- **Power Apps:** Build app logic, control visual properties like colors, and handle user interface behavior in canvas apps and model-driven command ribbons.
- **Microsoft Dataverse:** Create server-side calculated columns, rollup fields, and custom reusable environment functions.

- **Microsoft Copilot Studio:** Set variable values, parse text strings, and write conditional expressions for conversational agents.
- **Power Automate:** Implement reusable data-handling functions shared between cloud flows and apps. [1, 2, 3, 4, 5, 6]

Development and Pro-Code Tools

- **Visual Studio Code:** Write and edit formulas stored directly in human-readable YAML source files.
- **GitHub / Azure DevOps:** Manage, version control, and collaborate on Power Fx logic using standard code repositories.
- **Custom Web Hosts:** Embed the open-source Power Fx engine and formula bar components into independent web applications. [1, 2, 3]

Microsoft Power Fx overview - Power Platform | Microsoft Learn

Power Fx is the low-code language that Microsoft Power Platform uses. It's a general-purpose, strongly typed, declarative, and functional programming language.

Power Fx is expressed in human-friendly text. It's a low-code language that makers can work with directly in an Excel-like formula bar or Visual Studio Code text window.

- What if you could build an app as easily as you build a worksheet in Excel?
- What if you could take advantage of your existing spreadsheet knowledge?

Connector, MCP, Claude, Foundry

Absolutely. Let's make MCP as boring and practical as possible.

A lot of MCP discussions sound like:

"AI agents dynamically orchestrating heterogeneous tool ecosystems..."

But the skinniest real-world example is much simpler.

Without MCP

Your app needs SharePoint files.

You write:

```
App
├─ OAuth login
├─ Graph token handling
├─ Refresh tokens
```

```
5    ├─ Graph API calls
6    ├─ Error handling
7    ├─ SharePoint logic
8    └─ File retrieval
```

Lots of plumbing.

With MCP

Your app says:

```
1    "Get file QBR.pptx from SharePoint"
```

The MCP server handles:

```
1    OAuth
2    Connection
3    Permissions
4    API calls
5    Responses
```

The app only focuses on business logic.

Minimal Flow

```
1    User
2    ↓
3    App
4    ↓
5    MCP Client
6    ↓
7    SharePoint MCP Server
8    ↓
9    Microsoft Graph
10   ↓
```

```
11  SharePoint
```

Response comes back:

```
1  File
2  Metadata
3  Content
```

Then:

```
1  Connection ends
```

Done.

Tiny Example

Imagine a sales dashboard.

User clicks:

```
1  Show latest forecast
```

App code:

```
1  forecast = mcp.call_tool(
2      "sharepoint_get_file",
3      path="/Sales/Forecast.xlsx"
4  )
5
6  render_forecast(forecast)
```

The app doesn't know:

- Graph
- SharePoint APIs
- OAuth token flow

The MCP server does.

Where Entra ID Fits

The user logs in once:

```
1  Login → Entra ID
```

Entra issues token.

MCP server receives token.

MCP server can now call:

```
1  Microsoft Graph
```

as the user.

So:

```
1  Tim
2    ↓
3  App
4    ↓
5  MCP
6    ↓
7  Graph
8    ↓
9  SharePoint
```

The permissions are still enforced.

If Tim can't see the file:

```
1  Access Denied
```

MCP doesn't magically bypass security.

Tiny Foundry Version

Now insert Claude or GPT.

User types:

Summarize last month's project status.

Flow:

```
1  User Prompt
2    ↓
```

```
3  Foundry Agent
4      ↓
5  Claude reasoning
```

Claude decides:

```
1  Need project reports
```

Calls MCP tool:

```
1  sharepoint_search_reports()
```

Gets documents.

Then:

```
1  summarize()
```

Returns:

3 projects on track 1 delayed Budget variance 4% Top risk is vendor delivery

The model never directly knew SharePoint.

It simply knew:

```
1  A tool exists
2  Use it
```

Super Skinny Pseudo-Code

```
1  # User already signed in via Entra
2
3  question = "Summarize project status"
4
5  docs = mcp.call_tool(
6      "sharepoint_get_documents",
7      folder="/Projects/Status"
8  )
9
10 answer = foundry_model.chat(
```

```
        prompt=question,
        context=docs
    )

    return answer
```

That's honestly about as small as the idea gets.

The Mental Picture

Think of MCP as:

```
USB for enterprise data
```

Before USB:

```
Custom cable
Custom driver
Custom port
```

After USB:

```
Plug in
Use device
```

Before MCP:

```
Custom Graph code
Custom Salesforce code
Custom Jira code
Custom SAP code
```

After MCP:

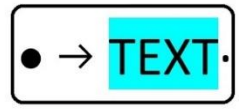
```
Tool available
Call tool
Get result
```

And Foundry agents/Claude become the thing smart enough to decide:

"For this question I should use the SharePoint tool, then the Teams tool, then the Outlook tool."

while Entra ID and Graph ensure all access stays inside the permissions the logged-in user already has. That's why Microsoft shops are paying attention: it potentially reduces a lot of integration code while keeping the security model they already trust.

Code, Syntax, Text: First UPL



Tim, I actually think your **"TEXT is the First UPL (Universal Programming Language)"** idea maps surprisingly well to where agents, MCP, Foundry, and modern IDEs appear to be heading.

The image itself appears to show:

- A box with an arrow pointing toward the word **TEXT**
- The statement:
  - **TEXT: the first UPL (Universal Programming Language)**
- A second statement:
  - **DO C.R.U.D. with Content on the Web**
- A small "Td Brand" style logo mark

The central idea I'm seeing is:

1	Intent
2	↓
3	TEXT
4	↓
5	Execution Environment

rather than:

1	Human
2	↓
3	Specific Language
4	↓
5	Specific SDK
6	↓
7	Specific API

Where Does the "Super Skinny Pseudo-Code" Run?

The answer is:

Anywhere that can interpret the TEXT.

The runtime is separate from the intent.

For example:

```
1  "Get project status from SharePoint
2  Summarize it
3  Create a report"
```

could execute in:

- Foundry Agent
- Claude Desktop
- VS Code
- Copilot Studio
- Azure Function
- Local Python script
- Power Automate

The TEXT is the request.

The execution layer decides:

```
1  What tools are needed?
2  What credentials are needed?
3  What APIs are needed?
```

MCP as an Execute-As Service

I think your phrase:

Giant Execute As Answer/Service

is actually not bad.

Imagine:

```
1  User
```

```
2  ↓
3  Intent
4  ↓
5  Agent
6  ↓
7  Execute-as-user
8  ↓
9  Systems
```

The entire point of:

- Entra ID
- OAuth
- Graph
- MCP

is that the execution occurs with the identity of the user.

So an agent is often doing:

```
1  Answer =
2  Result of executing actions
```

rather than:

```
1  Answer =
2  Text generated from training
```

Huge difference.

Foundry's Role

In a Microsoft shop I'd draw it this way:

```
1  Foundry
2  =
3  Agent Host
```

```
1  Claude/GPT
```

```
2   =
3   Reasoning Engine
```

```
1   MCP
2   =
3   Tool Interface
```

```
1   Entra ID
2   =
3   Identity
```

```
1   Graph
2   =
3   Microsoft Data Access Layer
```

```
1   M365
2   =
3   Actual Data
```

If TEXT Is The UPL

Then the stack becomes interesting.

Instead of:

```
1   Python
2   C#
3   JavaScript
4   PowerShell
5   SQL
```

you start with:

```
1   TEXT
```

Example:

```
1   Find all project updates
2   from the last 30 days.
```

```
3
4  Summarize risks.
5
6  Create a PowerPoint.
7
8  Save in SharePoint.
```

That's your source.

Now something must convert intent into execution.

The New Compiler

Historically:

```
1  Source Code
2  ↓
3  Compiler
4  ↓
5  Machine Code
```

Your model becomes:

```
1  TEXT
2  ↓
3  Reasoning Engine
4  ↓
5  Tool Selection
6  ↓
7  Execution
```

Almost:

```
1  Natural Language Compiler
```

What Parts Exist?

If we minimize tools and maximize automation:

1. 1. Editor

Maybe:

- VS Code
- Claude Desktop
- Copilot
- Browser

1. 2. Reasoning Layer

Claude/GPT

Reads TEXT.

Determines:

1. What
2. Where
3. How

1. 3. Tool Registry

MCP

Provides:

1. SharePoint Tool
2. Teams Tool
3. SQL Tool
4. Filesystem Tool
5. GitHub Tool

1. 4. Identity Layer

Entra ID

Determines:

1. Can Tim access this?

1	5. Execution Layer

Could be:

- Azure Functions
- Containers
- Agent Service
- Local Machine

This is where actions actually happen.

1	6. Result Layer

Outputs:

- Email
- PPTX
- DOCX
- JSON
- Dashboard
- Web Page

Your Mechanic Analogy

I really like:

Put less tools in the mechanic's hand.

Today's developer:

1	Needs to know
2	Graph
3	OAuth
4	REST
5	SDKs
6	Languages

```
7 Frameworks
```

Your future-state vision is:

```
1 Mechanic writes TEXT
```

Example:

```
1 Monitor the sales folder.
2
3 When a new spreadsheet arrives:
4
5 Calculate variance.
6
7 Email leadership.
8
9 Store summary.
10
11  Update dashboard.
```

The system figures out:

```
1 Power Automate?
2 Python?
3 Azure Function?
4 Graph?
5 SQL?
```

The Extreme Version

The really radical idea is this:

```
1 TEXT
2 is not documentation.
3 TEXT
4 is the application.
```

Meaning:

```
1  Application
2     =
3  Intent
```

not

```
1  Application
2     =
3  Compiled Code
```

Foundry Agents, MCP, Claude, Copilot Studio, LangGraph, Semantic Kernel, and similar technologies all feel like early steps toward that direction:

```
1  Human Intent
2     ↓
3  TEXT
4     ↓
5  Agent
6     ↓
7  Tool Selection
8     ↓
9  Execution
10    ↓
11 Outcome
```

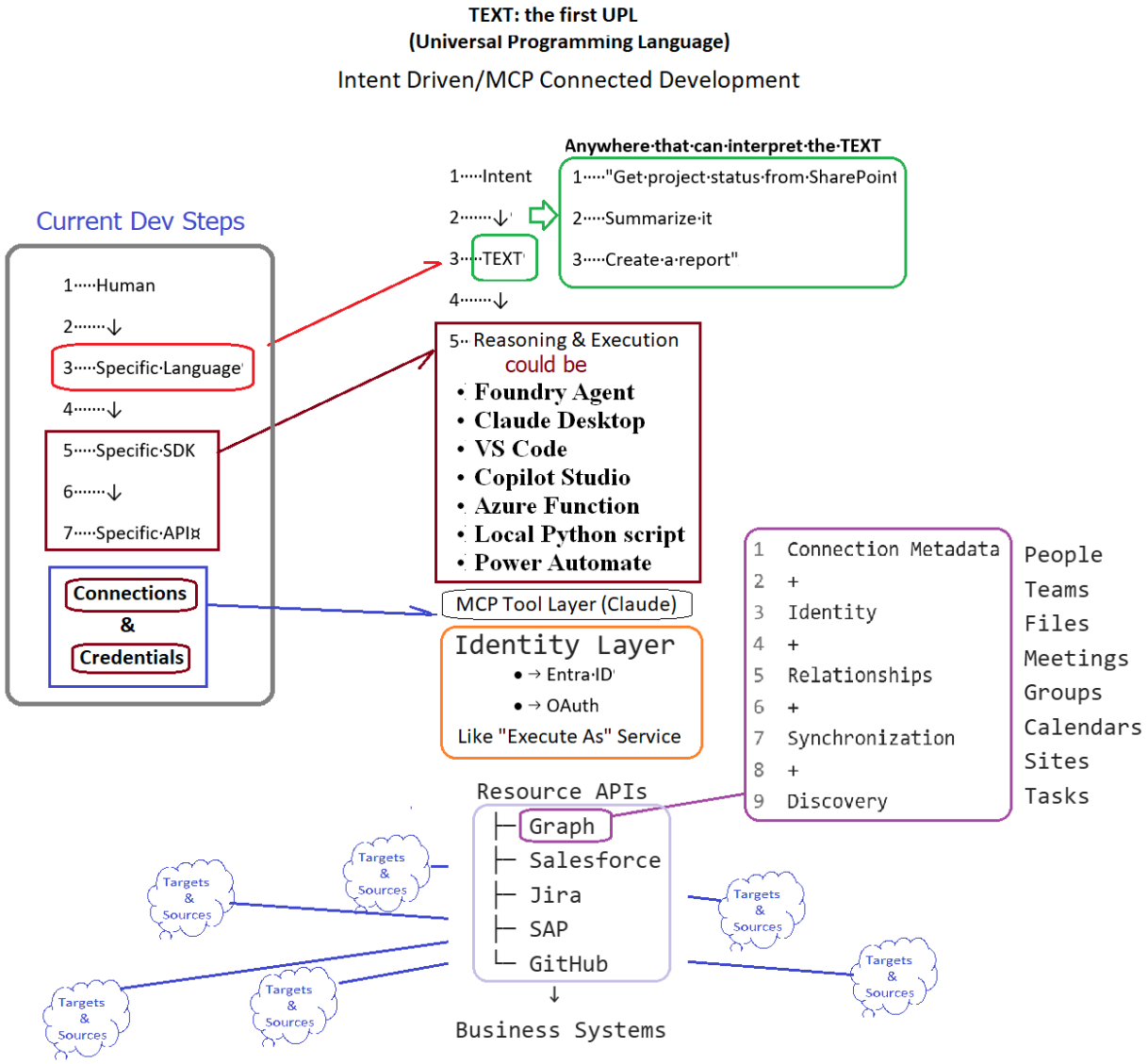
In that mental model, VS Code becomes less of a "coding editor" and more of an **intent editor**, while MCP becomes a standardized way for that intent to reach whatever tools, services, and enterprise systems are required to make the intent real.

Intent Driven/MCP Connected Development Image

"Intent becomes the source artifact. Identity enables access. MCP exposes tools. Execution environments determine implementation."

Intent defines the outcome. Identity grants authority. MCP provides capability. Execution determines implementation.

- MCP is the universal tool connector.
- Graph is the Microsoft organizational knowledge graph exposed as APIs.
- The execution layer is whatever runtime turns TEXT into actions against those connected systems.



"Intent becomes the source artifact. Identity enables access. MCP exposes tools. Execution environments determine implementation."

Modern Layers

I think you're onto something important, but I'd separate **"Execution Layer"** from **"Resource APIs."**

What you're describing is closer to:

```
TEXT
 ↓
Reasoning
 ↓
Execution
 ↓
Resource APIs
 ↓
Systems
```

Where:

Execution Layer

The thing that actually runs the work.

Examples:

- Foundry Agent
- Copilot Studio Agent
- Azure Function
- Power Automate
- Claude Desktop
- VS Code extension
- Python runtime

Its job is:

```
Receive Intent
Execute Steps
Manage State
Call Tools
Handle Results
```

Resource APIs

The doors into systems.

Examples:

```
Microsoft Graph
Salesforce API
Jira API
```

```
SAP API
GitHub API
ServiceNow API
```

Their job is:

```
Read
Create
Update
Delete
Search
```

within those systems.

Interesting Observation About Graph

I think where you're headed is:

> Graph feels different than a normal API.

And I'd agree.

Salesforce API mostly represents:

```
Salesforce
```

Jira API represents:

```
Jira
```

SAP API represents:

```
SAP
```

But Microsoft Graph is more like:

```
Enterprise Relationship Layer
```

or

```
Enterprise Object Graph
```

Instead of:

```
Get Customer
```

Graph often looks like:

```
User
  owns
    Files

User
  belongs to
    Groups
```

```
8
9  Group
10    has
11      Teams
12
13  Team
14    contains
15      Channels
16
17  Channel
18    contains
19      Messages
20
21  Message
22    references
23      Files
```

Everything is connected.

Hence the name:

```
1  Graph
```

Why Graph Feels "Next Gen"

You wrote:

> details returned from connections & synchronizing

That's actually pretty close to how I would explain it.

Graph is not merely:

```
1  REST Endpoint
```

It's closer to:

```
1  Unified Enterprise Knowledge Map
```

for Microsoft 365.

Think:

```
1  Outlook
2  OneDrive
3  SharePoint
4  Teams
5  Users
6  Groups
7  Planner
8  Calendar
```

all exposed through one model.

Your "Graph Rename" Thought

This is the part I find most interesting.

You seem to be viewing Graph as:

```
Connection Metadata
+
Identity
+
Relationships
+
Synchronization
+
Discovery
```

rather than simply:

```
API
```

And honestly, that's not a bad mental model.

When agents become common, Graph becomes less:

```
"an API developer calls"
```

and more:

```
"a map of organizational knowledge
and relationships"
```

The agent doesn't care whether data came from:

- Teams
- SharePoint
- Outlook
- OneDrive

The agent cares that:

```
Tim
 ↕
Project
 ↕
Files
 ↕
Meetings
 ↕
Emails
```

```
    ↕
Tasks
```

can be traversed.

That's very graph-like thinking.

So if I were refining your architecture, I'd say:

> **MCP is the universal tool connector. Graph is the Microsoft organizational knowledge graph exposed as APIs. The execution layer is whatever runtime turns TEXT into actions against those connected systems.**

That's remarkably close to the direction the industry seems to be heading.

CMS Interpretation

You also mentioned:

> CMS systems basically (UI/Data/Logic)

I'd broaden that slightly.

Most enterprise systems are really:

```
UI
+
Data
+
Business Rules
```

Examples:

Salesforce

```
CRM UI
Customer Data
Sales Logic
```

Jira

```
Issue UI
Issue Data
Workflow Logic
```

SAP

```
Enterprise UI
Business Data
ERP Logic
```

Microsoft 365

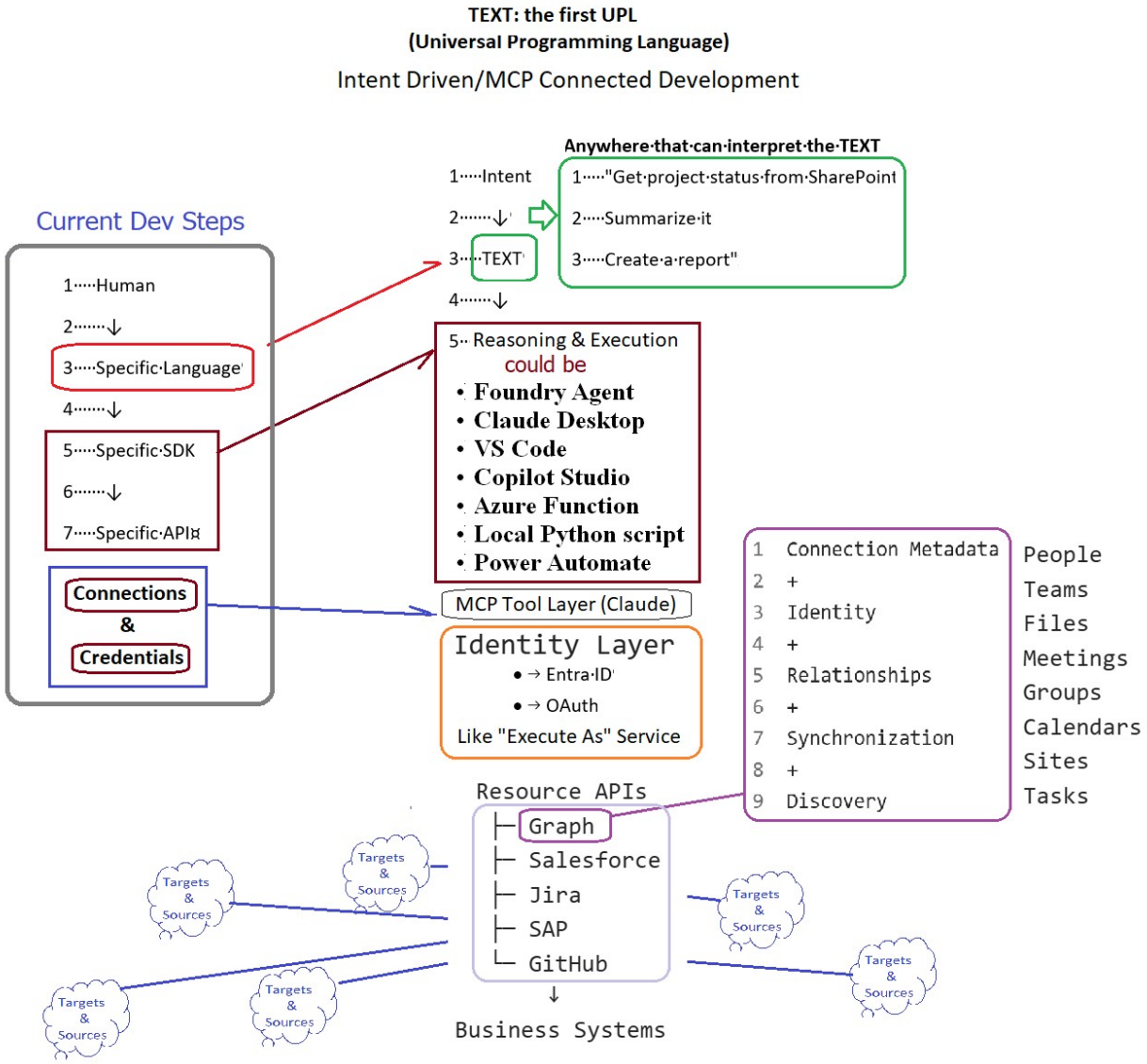
```
Teams UI
SharePoint UI
Document Data
```

```
4  Collaboration Logic
```

The APIs expose portions of those platforms.

Shifts in Development with AI w/Grok & Copilot + Image

Yo Grok - Happy Sunday.. Does this image seem to reflect the changes in modern dev today - obviously TEXT is not an official name..



Yes — this diagram captures several real shifts happening in modern software development right now, even if "TEXT / UPL" is a conceptual label rather than an official standard.What aligns well with current trends

- Intent over implementation details: Moving from "write code in language X using SDK Y against API Z" to expressing high-level goals ("Get project status from SharePoint → Summarize it → Create a report") is exactly the direction of agentic AI, copilots, and natural-language-to-action

systems. Tools like Claude, GitHub Copilot, Microsoft Copilot Studio, Azure AI agents, and various "Foundry"-style platforms are already doing this.

- MCP (Model Context Protocol): The diagram correctly highlights MCP as a tool/connection layer. Anthropic's MCP (and similar emerging protocols) is designed so AI models can discover and use tools, data sources, and services in a standardized way instead of hard-coding every integration. That's a genuine architectural change.
- Identity & "execute as" layer: Separating credentials/connections from the intent, and using modern identity (Entra ID / Azure AD, OAuth, service principals, managed identities) is how production systems are evolving. Secure, auditable "act on behalf of" patterns are critical once AI agents start touching real business systems.
- Unified resource access: Treating Microsoft Graph, Salesforce, Jira, SAP, GitHub, etc. as first-class resources behind a common layer, with discovery, relationships, and synchronization, matches the push toward composable, API-first platforms and data fabrics.
- Multiple possible execution environments: The list (Claude Desktop, VS Code, Power Automate, Azure Functions, local scripts, agent runtimes) reflects the reality that the same intent can be realized in different runtimes depending on context, governance, or scale.

What's still aspirational or idealized

- A single universal "TEXT" artifact that any interpreter can reliably turn into production-grade, secure, maintainable outcomes is not fully here yet. Current systems still require careful prompt engineering, tool definitions, guardrails, evaluation, and often fallback to traditional code.
- The diagram shows a clean separation and flow; in practice there is still a lot of glue, testing, observability, and human oversight needed.
- "Anywhere that can interpret the TEXT" is the goal of agent platforms, but reliability, determinism, cost, and security vary widely across those environments.

Bottom lineThe core thesis in the footer is accurate for the direction of travel:

"Intent becomes the source artifact. Identity enables access. MCP exposes tools. Execution environments determine implementation."

We're in the middle of that transition. Traditional language → SDK → API stacks aren't disappearing, but they are increasingly being wrapped or generated by higher-level intent + agent systems. Your diagram is a solid, forward-looking illustration of that shift.Happy Sunday — interesting mental model!

Speaking of agents - an on behalf of - that can execute as - and do CRUD Ops - with a reason(ing - aka filter on steroids - aka machine learning in a horrible nutshell) - where does clever nomenclature in marketing - hide a templated service that crawls data sources and returns results?

Most of the time it hides behind names like "agent," "copilot," "reasoning engine," "autonomous workflow," or "intelligent action layer."What's actually running under the hood is usually a fairly templated pipeline:

1. Crawl / index / connector layer
   Pre-built or lightly configured connectors that periodically (or on-demand) pull data from SharePoint, Graph, Salesforce, Jira, SAP, GitHub, file shares, databases, etc. Classic enterprise search / Graph connectors / Azure AI Search / Elasticsearch-style indexing dressed up with extra metadata.

2. Retrieval + filter ("reasoning")
   Semantic search, vector retrieval, keyword + ML ranking, or a lightweight LLM call that re-ranks or summarizes the retrieved chunks. This is the part marketing loves to call "reasoning," "cognitive filtering," or "machine learning on steroids." In practice it's mostly retrieval-augmented generation (RAG) or advanced search with an LLM wrapper.

3. Identity / "execute as" / on-behalf-of
   Delegated OAuth, Entra ID / service principal, managed identity, or "Execute As" patterns so the service can act with the user's (or a service account's) permissions. This is real and important, but it's the same identity plumbing that has existed for years in enterprise apps.

4. CRUD surface
   Thin action layer that turns the retrieved results (or LLM-suggested changes) into Create / Read / Update / Delete calls against the same APIs the connectors already know. Often implemented as pre-defined tool schemas (exactly what MCP and similar protocols formalize).

5. Templated orchestration
   A low-code or prompt-templated flow (Power Automate, Copilot Studio, Azure Logic Apps, LangChain-style agent loops, Foundry agents, etc.) that stitches the above together. The "agent" decides which tools to call and in what order, but the tools themselves are mostly fixed templates.

Where the clever nomenclature does the heavy lifting

- "Agent" or "Agentic AI" → usually means "LLM + tool-calling loop over a fixed set of connectors."
- "Reasoning" → retrieval + ranking + a bit of chain-of-thought or tool selection. Rarely deep multi-step planning or novel inference.
- "On behalf of / execute as" → standard delegated identity, just highlighted because agents make the permission surface scarier.
- "CRUD with intelligence" → the intelligence is mostly in deciding what to CRUD and which records; the actual write operations are ordinary API calls.

You'll see this exact pattern in:

- Microsoft 365 Copilot / Copilot Studio agents (Graph connectors + semantic index + LLM)
- Claude Desktop / MCP tool servers
- Salesforce Einstein / Agentforce

- Many "enterprise AI agent" platforms that advertise "connect to all your data sources and take action"

The diagram you shared earlier is actually quite honest about the pieces (Intent → MCP tool layer → Identity → Resource APIs).

The marketing version just collapses all of that into a single shiny noun and implies far more autonomy and novel intelligence than the underlying templated crawl → retrieve → act service usually delivers.

So yes — a large fraction of today's "on-behalf-of agents that reason and do CRUD" are sophisticated, well-integrated, permission-aware search-and-action services wearing an agent costume.

The costume is useful (better UX, standardized tool interfaces, identity handling), but the core engine is still crawl + filter + template.

Outlook, Teams Discussions

Considerations

- Greetings
- Intent Collection
- Clarification
- Escalation
- Follow-Up

Outlook Example

Flow: Vendor Invoice Received

Business Scenario

Contoso receives invoices by email.

Trigger

```
When a new email arrives
```

Actions

```
New Email
    ↓
Check Subject Contains "Invoice"
    ↓
Save Attachment
    ↓
Post Message to Teams
    ↓
Notify Finance
```

Tools

- Outlook Trigger
- OneDrive or SharePoint

- Teams Connector

Student Learning

- Triggers
- Conditions
- Attachments
- Variables
- Teams Integration

Why It Works

Students immediately understand:

> "Email starts a business process."

This also becomes the starting point for your Day 2 Contoso scenario.

Outlook Example (Slightly More Advanced)

Flow: Email to Desktop Flow

```
New Email
     ↓
Save Attachment
     ↓
Run Desktop Flow
     ↓
Enter Data into Legacy System
```

Connection to PL-500

Students see:

```
Outlook
     ↓
Cloud Flow
     ↓
Desktop Flow
```

This bridges nicely from legacy RPA.

Teams Example

Flow: Approval Request

Business Scenario

Manager approval is required before paying an invoice.

Trigger

```
Manual Button
```

or

```
New Invoice Submitted
```

Actions

```
Create Approval
      ↓
Send Teams Approval Card
      ↓
Manager Approves
      ↓
Post Result to Teams
```

Tools

- Power Automate
- Teams
- Approvals

Student Learning

- Human-in-the-loop workflows
- Approvals
- Adaptive Cards
- Decision branches

Teams Example (My Favorite)

Flow: Ask for PTO Approval

```
Microsoft Form
        ↓
Power Automate
        ↓
Teams Approval
        ↓
Approved?
   ↙        ↘
Yes          No
 ↓            ↓
Notify       Notify
Employee     Employee
```

Why It Works

Everyone understands leave requests.

No special business knowledge required.

Easy to demo.

Introduces:

- Forms
- Power Automate
- Teams
- Approvals

in one exercise.

Outlook Lab

"Invoice Email Processing"

```
Outlook
   ↓
Attachment
   ↓
SharePoint
   ↓
Notification
```

Teams Lab

"Manager Approval Workflow"

```
Invoice
   ↓
Approval
   ↓
Teams Card
   ↓
Approve / Reject
```

Introduce Copilot Studio, you can show the evolution:

Traditional

```
Outlook
   ↓
Flow
   ↓
Teams Approval
```

Modern

```
User
   ↓
Agent
```

```
    ↓
Flow
    ↓
Teams Approval
    ↓
Response
```

That "same process, modern front-end" story tends to make the light bulb come on for students moving from RPA to AI Agents.

Governance, Security, Agents Conversation Topic (d4)

Some of these seem to be a human business need vs a setting or software feature that gets configured or a library of functions (Actions) that is not the library but knowing which function & what make it work.

Note: Terms or Phrase may seem repetitive in behavior or discussion for humans but may only have a few software settings that configure the environment.

Governance Topics

- Environment Strategy
- DLP Policies
- Monitoring
- Publishing

Security Topics

- Authentication
- Authorization
- Dataverse Roles
- Connector Permissions

Microsoft courses often become a bit abstract because some topics are:

- **Business concepts** (Conversation Design)
- **Configuration concepts** (Topics)
- **Content concepts** (Knowledge Sources)
- **Implementation concepts** (Actions)
- **Administrative concepts** (Governance)
- **Security concepts** (Security)

For a Power Automate audience:

> - Why do I need this?
> - What am I configuring?
> - What is the simplest example?

> - What makes it actually work?

Conversation Design

Business Need

Users need a natural way to interact with the Agent.

Without Conversation Design:

```
User:
Where is my invoice?

Agent:
I cannot understand your request.
```

With Conversation Design:

```
User:
Where is my invoice?

Agent:
Do you know the invoice number or vendor name?
```

What Are You Actually Configuring?

Not technology.

You are designing:

- Questions
- Clarifications
- Follow-up prompts
- Escalation paths

Think:

```
Help Desk Script
```

for an AI Agent.

Simplest Exercise

Design a conversation for:

> "Check Invoice Status"

Ask:

1. What should the agent ask first?
2. What if information is missing?
3. What if there are multiple invoices?

Topics

Business Need

Agents need to identify user intent.

What Is It?

A Topic equals:

```
One Business Task
```

Examples:

- Check Invoice Status
- Submit PTO
- Open Ticket
- Check Approval Status

What Are You Configuring?

Inside Copilot Studio:

```
Topic
     ↓
Trigger Phrases
     ↓
Questions
     ↓
Actions
```

Simplest Exercise

Create Topic:

```
Invoice Status
```

Trigger Phrases:

- Where is my invoice?
- Check invoice
- Invoice status

Knowledge Sources

Business Need

Users ask questions.

Agent needs answers.

What Is It?

Content the agent can search.

Examples:

- SharePoint
- Websites
- PDFs
- Dataverse

What Are You Configuring?

You connect information repositories.

```
SharePoint Site
      ↓
Agent
```

or

```
PDF
      ↓
Agent
```

Simplest Exercise

Upload:

```
Vendor Policy.pdf
```

Ask:

```
What is the approval limit?
```

Agent retrieves answer.

Actions

This is usually the most important topic for Power Automate .

Business Need

Agent must do more than answer questions.

Agent must perform work.

What Is It?

An Action allows the Agent to execute something.

Most commonly:

```
Call Power Automate Flow
```

What Are You Configuring?

Not the entire flow.

Just:

```
Agent Topic
      ↓
Action
      ↓
Flow
```

Simplest Exercise

Agent:

```
Check Invoice Status
```

Action:

```
Get Invoice Record
```

Flow:

```
Search Dataverse
```

Result:

```
Invoice Approved
```

Instructor Message

For PL-500 :

```
Flow = Engine

Agent = Dashboard
```

The flow still does the work.

Governance

Business Need

Who is allowed to create AI?

Who is allowed to publish it?

What Is It?

Administrative control.

Examples:

- DLP Policies
- Environments
- Publishing Rules
- Monitoring

What Are You Configuring?

Mostly Admin Center settings.

Simple Exercise

Discussion:

Who should be allowed to publish an Agent?

```
Everyone?

Only IT?

Business Unit Owners?
```

Much more valuable than button-clicking.

Security

Business Need

The agent should only see data the user is allowed to see.

What Is It?

Identity and permissions.

Examples:

- Authentication
- Roles
- Dataverse Permissions
- Connector Permissions

What Are You Configuring?

Mostly:

```
User
     ↓
Security Role
     ↓
Data Access
```

Simplest Exercise

Scenario:

```
Employee asks:
Show all payroll records.
```

Agent:

```
Access Denied
```

Discussion:

Why?

Because permissions follow the user.

Drill Through Summary Slide (d4)

```
Conversation Design
    ↓
How Agent Talks

Topics
    ↓
What Agent Understands

Knowledge Sources
    ↓
 What Agent Knows

 Actions
    ↓
 What Agent Does

 Governance
    ↓
 Who Can Use It

 Security
    ↓
 What Data It Can Access
```

For your audience, I'd spend the most time on:

1. **Actions** (40%)
2. **Knowledge Sources** (25%)
3. **Topics** (15%)
4. **Conversation Design** (10%)
5. **Governance** (5%)
6. **Security** (5%)

because Actions + Knowledge Sources are where Power Automate practitioners finally connect:

```
Agent
   ↓
Knowledge
   ↓
```

```
Action
   ↓
Flow
```

and that is usually the "aha!" moment that bridges legacy PL-500 automation to modern Copilot Studio architecture.

Mechanics of Agent Architecture (d4)

I like where you're heading. For an automation audience, I'd define these less as "features" and more as **mechanics of an Agent Architecture**.

Think of Day 4 as:

```
User
  ↓
Conversation
  ↓
Topic
  ↓
Knowledge
  ↓
Action
   ↓
 Flow
   ↓
 Business System
       ↑
 Security
       ↑
 Identity
```

Agent Architecture Components

How Agent Talks

Conversation Design

Purpose

Defines how the Agent interacts with humans.

Mechanic

Conversation Layer

```
User Question
      ↓
Agent Response
      ↓
Clarification
```

```
      ↓
Action or Answer
```

Think Of It As

- User Experience (UX)
- Chat Design
- Help Desk Script

Tool

- Copilot Studio

Example

```
User:
Where is my invoice?

Agent:
Please provide an invoice number
or vendor name.
```

What Agent Understands

Topics

Purpose

Maps user intent to business functions.

Mechanic

Intent Recognition Layer

```
Topic:
Invoice Status

Possible User Phrases:

- Where is my invoice?
- Check invoice
- Has invoice been approved?
```

Think Of It As

Routing table.

```
User Intent
      ↓
Correct Business Function
```

Tool

- Copilot Studio Topics

Example Topics

- Invoice Status
- PTO Requests
- Open Ticket
- Approval Status

What Agent Knows

Knowledge Sources

Purpose

Provides information for grounded responses.

Mechanic

Knowledge Layer

```
Question
    ↓
Knowledge Source
    ↓
Answer
```

Think Of It As

The Agent's library.

Tool

- SharePoint
- Dataverse
- Websites
- PDF Documents

Example

```
Vendor Policies PDF
        ↓
Agent Answers
Approval Rules
```

What Agent Does

Actions

Purpose

Allows an Agent to perform work.

Mechanic

Execution Layer

```
Question
    ↓
Agent
    ↓
Action
    ↓
Power Automate Flow
```

Think Of It As

Agent capabilities.

Not knowledge.

Not conversation.

Actual work.

Examples

- Create Request
- Get Status
- Create Record
- Trigger Approval
- Run Flow

Power Automate Audience Translation

```
Flow = Engine

Action = Button

Agent = Dashboard
```

The Action connects the conversation to the automation.

Who Can Use It

Identity

Purpose

Verifies who the user is.

Mechanic

Authentication Layer

```
User
   ↓
Microsoft Entra ID
   ↓
Authenticated Identity
```

Think Of It As

Login.

"Who are you?"

Technology

- Microsoft Entra ID
- OAuth
- SSO
- Conditional Access

What Data It Can Access

Authorization

Purpose

Controls what the Agent may access.

Mechanic

Authorization Layer

```
Authenticated User
          ↓
RBAC
          ↓
Data Access
```

Think Of It As

"What are you allowed to see?"

Technology

- Dataverse Security Roles
- RBAC
- SharePoint Permissions
- Connector Permissions

Example

```
Employee

Cannot Read Payroll

Manager

Can Read Payroll
```

The Agent inherits permissions rather than bypassing them.

How the Agent Reaches Systems

Connectivity Layer

This is where MCP is becoming an interesting discussion.

Mechanic

Connection Layer

```
Agent
   ↓
Connector
   ↓
Flow
   ↓
Application
```

Today this is usually:

- Power Automate Connectors
- Dataverse
- APIs
- Custom Connectors

Emerging pattern:

```
Agent
   ↓
MCP Server
   ↓
External System
```

MCP Translation

Model Context Protocol (MCP) acts like a standardized tool interface.

Think:

```
Knowledge Source
      +
```

```
Action Endpoint
      +
Security Context
```

packaged as a reusable service.

For training purposes I'd keep it simple:

> MCP is to AI Agents what Connectors are to Power Automate.

Not exactly the same thing, but close enough conceptually for students.

Day 4 Architecture Slide

```
User
  │
  ▼
Identity
(Who Are You?)
  │
  ▼
Conversation
(How Agent Talks)
  │
  ▼
Topic
(What Agent Understands)
  │
  ▼
Knowledge
(What Agent Knows)
  │
  ▼
Action
(What Agent Does)
  │
  ▼
Power Automate Flow
  │
  ▼
Business System
  ▲
  │
Authorization
(What Data Can Be Accessed?)
```

That's probably the cleanest "mechanic" view of Day 4 for a Power Automate audience because every component answers a practical question rather than sounding like a product feature list.

The difference between: Concepts & Product Configuration

> **Conceptual Architecture** ("What Agent Understands")

and

> **Product Mechanics** ("Where do I actually configure it?")

For instructor-led training, students need both.

I'd add a slide like this after the Day 4 overview.

Agent Architecture Components and Where They Live

Architecture Layer	Purpose	Product Area	What Gets Created?	Stored Where?
Conversation	How Agent Talks	Copilot Studio	Instructions, prompts, responses	Agent configuration
Topic	What Agent Understands	Copilot Studio Topics	Trigger phrases, conversation branches	Agent Topic definitions
Knowledge Source	What Agent Knows	Copilot Studio Knowledge	SharePoint, PDFs, Websites, Dataverse	Connected sources
Action	What Agent Does	Copilot Studio Actions	Flow calls, API calls, connectors	Agent Actions
Flow	Executes Work	Power Automate	Workflow logic	Cloud Flow
Desktop Flow	Legacy Automation	Power Automate Desktop	UI automation	Desktop Flow
Identity	Who Can Use It	Entra ID	Users, Groups, OAuth	Entra ID
Authorization	What Can Be Accessed	Dataverse / M365 Security	Roles & Permissions	Security Layer

Think Like a Power Automate Builder

Most Power Automate students understand:

```
Trigger
 ↓
Action
 ↓
Result
```

Agents introduce three new concepts:

```
Conversation
Topic
Knowledge
```

These can feel abstract until you explain where they are configured.

Conversation

Business View

How the Agent talks.

Product View

Configured inside:

```
Copilot Studio
    ↓
Agent
    ↓
Instructions
```

Examples:

- System prompt
- Greeting
- Clarification wording
- Escalation wording

Stored As

Part of the Agent definition.

Think:

```
Help Desk Script
```

Topics

Business View

What the Agent understands.

Product View

Configured inside:

```
Copilot Studio
    ↓
Topics
```

Example:

```
Topic:

Check Invoice Status
```

Trigger Phrases:

```
Where is my invoice?
Invoice status
Did it get approved?
```

Stored As

Agent Topic objects.

Think:

```
Intent Library
```

Students usually understand Topics when you describe them as:

> Flow triggers for conversations.

Knowledge Sources

Business View

What the Agent knows.

Product View

Configured inside:

```
Copilot Studio
    ↓
Knowledge
```

Add:

- SharePoint
- Website
- PDF
- Dataverse

Stored As

Knowledge Connections

Think:

```
Searchable Library
```

Actions

Business View

What the Agent can do.

Product View

Configured inside:

```
Copilot Studio
    ↓
Actions
```

Most common:

```
Call Power Automate Flow
```

Stored As

Action definitions

Think:

```
Agent Skills
```

The Modern Power Platform Architecture

```
        User
          │
          ▼

  Conversation Layer
  (How Agent Talks)

          │
          ▼

      Topics
  (What Agent Understands)

          │
          ▼

  Knowledge Sources
   (What Agent Knows)

          │
          ▼

      Actions
    (What Agent Does)

          │
```

```
          ▼

    Power Automate
     (Execution)

          |
          ▼

  Business Systems
```

Admin View (The Setup Locations)

This is the slide I think your audience will love:

I Need To...	Product
Create Users	Entra ID
Assign Permissions	Dataverse Security Roles
Build Agent	Copilot Studio
Add Topics	Copilot Studio
Add Knowledge	Copilot Studio
Add Actions	Copilot Studio
Build Workflow	Power Automate
Build RPA	Power Automate Desktop
Create Business Data	Dataverse
Publish Agent	Copilot Studio
Govern Agent	Power Platform Admin Center

That turns the "ghostly" concepts into tangible product locations.

Bonus Slide: Agent ≈ Flow Analogy

Power Automate	Agent Architecture
Trigger	Topic
Variables	Context
Connector	Action
Data Source	Knowledge Source
Flow Logic	Agent Reasoning
Cloud Flow	Execution Engine
User Form	Conversation

This slide often creates the *aha moment* because Power Automate users realize:

> Topics are conversation triggers.
>
> Knowledge Sources are conversational data sources.
>
> Actions are agent connectors.

> Flows are still the engine underneath.

That framing makes Day 4 much less theoretical and much more "I know where to click and what to build."

End

Massively Loose Notes

Portals

https://admin.powerplatform.microsoft.com/home

Setting up Dataverse…

- https://admin.powerplatform.microsoft.com/home
- https://make.powerautomate.com/
- https://m365.cloud.microsoft/

Power Automate = cloud orchestration.

Power Automate for desktop = the RPA robot.

capacity for vm

https://learn.microsoft.com/en-us/power-automate/desktop-flows/capacity-utilization-process

Links that pop up in labs

https://learn.microsoft.com/en-us/training/modules/build-first-desktop-flow/

Record with Copilot function. Recording with Copilot works by recording actions you take on your desktop while listening to microphone input for clarifying instructions.

https://learn.microsoft.com/en-us/training/modules/build-first-desktop-flow/5-record-desktop-actions

https://learn.microsoft.com/en-us/power-automate/desktop-flows/actions-reference/mouseandkeyboard#sendkeys

add to Co to extend simple flow:

be sure a new notepad file is opened before Send keys

create a desktop flow that helps Contoso Coffee shop employees enter invoice information into the desktop management system.

Assets library allows you to include more functionality in desktop flows. For example, you can make certain cloud connectors available in your flow, upload custom actions to the assets library when required, or you can search for available UI elements collections.

https://learn.microsoft.com/en-us/power-automate/desktop-flows/assets-library

UI elements are elements that are grabbed and captured from the various user interfaces (either desktop applications or web pages). These elements can be text fields, buttons, links, or anything else that you can interact with in the target applications.

After these elements are captured, they can be associated with the respective UI or web automation actions, so that the corresponding interaction with the elements can be automated in the context of desktop flows.

UI elements collections now offer makers and admins the ability to have control and central management over "groups" of UI elements, which can be shared across multiple users and imported in multiple desktop flows as reusable components.

https://learn.microsoft.com/en-us/power-automate/desktop-flows/ui-elements-collections

Power Automate's automation center

https://learn.microsoft.com/en-us/power-automate/automation-center-overview

The process map makes troubleshooting and monitoring in Power Automate more efficient and transparent. It shows the main orchestrating flow and all its child flows invoked during a process run.

Reference Links & Exercises

1. Introduction to the Power Automate Hosted RPA - Power Automate | Microsoft Learn
2. Course AB-410T00-A: Build intelligent applications - Training | Microsoft Learn
3. AB-410T00-Build-intelligent-applications/Instructions/Labs at main · MicrosoftLearning/AB-410T00-Build-intelligent-applications
4. Create a Plan and Refine the Requirements - Training | Microsoft Learn
    a. Use plans in Power Apps to describe the problem in plain language and let AI agents handle the solution design.
5. Build and Manage Your Solution - Training | Microsoft Learn
6. Create a Plan from an Existing Solution - Training | Microsoft Learn
7. Exercise - Create a Microsoft Dataverse table - Training | Microsoft Learn
8. Exercise - Import data into your database - Training | Microsoft Learn
9. Exercise - Create table relationships - Training | Microsoft Learn
10. Exercise - Create a custom table and import data - Training | Microsoft Learn
11. Exercises - Training | Microsoft Learn
    a. Dataverse columns
12. Adding or disabling an environment user - Training | Microsoft Learn
13. Exercise - Create a custom role - Training | Microsoft Learn
14. Exercise - Create your first app in Power Apps - Training | Microsoft Learn
15. Exercise - Create an app from Excel using Copilot - Training | Microsoft Learn
16. Apply Power Fx Formulas to Canvas App Controls - Training | Microsoft Learn
17. Work with Power Fx formulas in canvas apps - Training | Microsoft Learn
    a. Programming part so to speak
    b. Use Copilot to write and understand Power Fx formulas

c. 

d. Variables - Store a value and reuse it across multiple controls or screens

18. Build reusable components and formulas in canvas apps - Training | Microsoft Learn
19. Forms overview - Training | Microsoft Learn
20. Get started with model-driven apps in Power Apps - Training | Microsoft Learn
21. Extend model-driven apps with custom pages - Training | Microsoft Learn
22. Exercise - Create a model-driven app - Training | Microsoft Learn
23. Build pages with generative AI - Training | Microsoft Learn
24. Exercise - Control security when sharing model-driven apps - Training | Microsoft Learn
    a. To get the most out of this exercise, you will first need to create an app as described in the previous unit of this module and create the Pet table as described in unit Create a Microsoft Dataverse table of Get started with Dataverse Service module.
    b. Contoso, which has a pet grooming business that services dogs and cats. An app that has a custom table for tracking the pet grooming business has already been created and published.
25. Incorporate business process flows - Training | Microsoft Learn
26. Add logic to commands with Power Fx - Training | Microsoft Learn
    a. Commanding lets makers add custom buttons to the command bar of a model-driven app using the command designer.
27. Enable Copilot chat for app users - Training | Microsoft Learn
    a. Copilot chat in Power Apps is being deprecated for model-driven apps in environments that are not enabled for Dynamics 365 apps.
28. Add agents to your model-driven app - Training | Microsoft Learn
    a. Model-driven apps can host agents that work alongside your users, automating tasks and surfacing work that needs human input — all without users leaving the app.
    b. Autonomous agents — *Created in Microsoft Copilot Studio and connected to the Power Apps MCP (Model Context Protocol) server.*
    c. These agents run in the background, take action on Dataverse data, and post tasks to the agent feed when they need human review or input.
    d. App assistant agents — Provide custom topics, knowledge sources, and conversational capabilities within the Copilot chat pane of the app.
    e. **When an agent needs a human to step in, it creates a task in the **agent feed** — a task list that appears at the top of the app's site map.
29. Configure forms, charts, and dashboards in model-driven apps - Training | Microsoft Learn
30. Introduction to Power Pages design studio - Training | Microsoft Learn
31. Secure your Power Pages site - Training | Microsoft Learn
32. Add an AI-powered agent to your site - Training | Microsoft Learn
    a. An AI-powered agent in Power Pages provides conversational support to your site's visitors and users. When you add an agent to your site, visitors can ask questions in natural language and receive AI-generated responses based on your site's content, without having to navigate through pages manually.

33. Exercise - Edit pages - Training | Microsoft Learn
34. Get started with Power Automate - Training | Microsoft Learn
35. Exercise - Create recurring flows - Training | Microsoft Learn
36. Exercise - Monitor incoming emails - Training | Microsoft Learn
37. Exercise - Share flows - Training | Microsoft Learn
    a. Create a shared flow by adding more owners to an existing flow. After new owners are added to a flow, the flow appears on the Shared with me tab
38. Troubleshoot flows - Training | Microsoft Learn
39. Convert a flow to an agent flow - Training | Microsoft Learn
40. Use Dataverse triggers and actions in Power Automate - Training | Microsoft Learn
41. Create, update, delete, and relate actions - Training | Microsoft Learn
42. Exercise - Create a cloud flow with a Dataverse connector - Training | Microsoft Learn
43. Build an approval request - Training | Microsoft Learn
    a. Start and wait for an approval action
44. Create a business process flow - Training | Microsoft Learn
45. Create AI Builder grounded prompts with Dataverse data - Training | Microsoft Learn
46. Exercise - Create a grounded prompt in AI Builder - Training | Microsoft Learn

End ref links